



**ÉCOLE MAROCAINE DES
SCIENCES DE L'INGÉNIEUR**

Membre de
HONORIS UNITED UNIVERSITIES



ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR
Membre de HONORIS UNITED UNIVERSITIES — Campus de Casablanca
Cycle Ingénieur — Filière **Génie Informatique et Réseaux**
3^{ème} année

RAPPORT DE PROJET DE FIN D'ANNÉE (PFA)

Stage réalisé au sein de **CIH BANK** — Direction des Systèmes d'Information

Automatisation de la gouvernance et de la conformité des dépôts GitLab

Conception d'un outil d'audit de conformité
et d'une chaîne CI/CD de provisionnement de projets

Projets AUDIT-TOOL et GOVERN X

Réalisé par :

Moukaouil Chouaib

Encadrant professionnel

M. Yassine NAJI
CIH Bank

Encadrant pédagogique

.....
EMSI Casablanca

Année universitaire 2025 – 2026

Période de stage : juillet – août 2026

Dédicace

*À mes parents,
pour leur soutien sans faille, leur patience et les sacrifices
consentis tout au long de mes années d'études.*

*À mes frères et sœurs,
pour leurs encouragements constants.*

*À mes enseignants de l'EMSI,
qui m'ont transmis bien plus que des connaissances techniques.*

*À toute l'équipe de la Direction des Systèmes d'Information de CIH Bank,
pour son accueil et sa confiance.*

*À tous ceux qui, de près ou de loin,
ont contribué à faire de cette expérience une réussite.*

Moukaouil Chouaib

Remerciements

Au terme de ce projet de fin d'année, je tiens à exprimer ma reconnaissance à l'ensemble des personnes qui ont rendu cette expérience possible et enrichissante.

Je remercie tout d'abord **CIH Bank**, et plus particulièrement la **Direction des Systèmes d'Information**, de m'avoir accueilli en son sein et de m'avoir confié un sujet réel, ancré dans les préoccupations quotidiennes des équipes, plutôt qu'un exercice académique déconnecté de la production.

Ma gratitude va tout spécialement à mon encadrant professionnel, **M. Yassine NAJI**, pour la qualité de son accompagnement tout au long du stage. Sa disponibilité, la précision de ses retours et son exigence méthodologique m'ont permis de progresser rapidement sur des sujets que je découvrais entièrement — l'API GitLab, l'intégration continue, la gouvernance des dépôts de code. La liberté qu'il m'a laissée dans les choix de conception, associée à un cadrage régulier des priorités, a été déterminante dans l'aboutissement des deux projets présentés dans ce rapport.

Je remercie également l'ensemble des collaborateurs de l'équipe DevOps pour leur disponibilité et leurs explications sur le fonctionnement réel de la plateforme GitLab interne, ainsi que le corps professoral et l'administration de l'**École Marocaine des Sciences de l'Ingénieur**, pour la formation dispensée durant ces années. Les fondamentaux acquis en algorithmique, en réseaux et en systèmes ont constitué le socle sur lequel ce travail a pu être bâti.

Enfin, je remercie ma famille et mes camarades de promotion, dont le soutien constant a accompagné chacune des étapes de ce parcours.

Résumé

Dans un établissement bancaire, le code source est un actif réglementé au même titre que les données qu'il manipule. Les règles qui encadrent son cycle de vie — qui peut pousser sur la branche de production, qui peut créer un tag de version, une fusion peut-elle avoir lieu sans chaîne d'intégration verte — constituent un référentiel de gouvernance que chaque dépôt doit respecter. Or, sur une instance GitLab comptant plusieurs dizaines de projets, la vérification manuelle de ce référentiel est coûteuse, non reproductible et, en pratique, rarement effectuée.

Ce projet de fin d'année, réalisé au sein de la Direction des Systèmes d'Information de CIH Bank, apporte une réponse en deux temps.

Le premier livrable, **Audit-Tool**, est un outil en ligne de commande écrit en Python qui parcourt l'ensemble des projets d'une instance GitLab et vérifie cinq règles de conformité : protection des branches `main`, `develop` et `release/*` avec les niveaux d'accès attendus, protection des tags de version avec détection automatique du *stack* à partir du manifeste de build, activation de la contrainte « *Pipeline must succeed* » sur les demandes de fusion, et gestion des accès d'un utilisateur de service. L'outil ne se limite pas au constat : une couche de remédiation corrige les écarts détectés, avec deux stratégies distinctes selon l'édition de GitLab — une limitation d'API découverte en cours de développement. Trois formats de rapport (CSV, JSON, HTML) sont produits et l'ensemble est industrialisé dans une chaîne GitLab CI.

Le second livrable, **GovernX**, déplace le curseur de la correction vers la prévention : plutôt que d'auditer des projets déjà dérivés, il garantit qu'un projet naît conforme. Il s'agit d'une chaîne CI/CD en trois étapes qui, à partir d'un simple formulaire, résout un dépôt modèle par *topic* GitLab, le recopie intégralement vers un ou plusieurs nouveaux projets, aligne l'identité du projet dans son manifeste de build, applique les *topics* de classification, puis réutilise les règles d'Audit-Tool pour auditer et corriger automatiquement le résultat. Le comportement est *fail-closed* : toute ambiguïté bloque la chaîne plutôt que de produire une configuration devinée.

Les deux projets totalisent environ 3 500 lignes de Python, 84 tests unitaires sans dépendance réseau, et ont été validés en conditions réelles sur l'instance GitLab de développement. Ce rapport décrit le contexte, les choix d'architecture, l'implémentation, les bugs marquants rencontrés — dont plusieurs faux positifs particulièrement instructifs — ainsi que les perspectives d'industrialisation.

Mots-clés : DevOps, DevSecOps, GitLab, API REST, Python, CI/CD, gouvernance, conformité, automatisation, *Infrastructure as Code*.

Abstract

In a banking institution, source code is a regulated asset in the same way as the data it processes. The rules governing its lifecycle — who may push to the production branch, who may create a release tag, whether a merge can happen without a green pipeline — form a governance baseline that every repository must comply with. On a GitLab instance hosting dozens of projects, manually checking that baseline is expensive, non-reproducible and, in practice, rarely done.

This final-year project, carried out within the IT Department of CIH Bank, addresses the problem in two stages. **Audit-Tool** is a Python command-line tool that scans every project of a GitLab instance, verifies five compliance rules, then automatically remediates the deviations it finds, with two distinct strategies depending on the GitLab edition. It produces CSV, JSON and HTML reports and runs as a scheduled GitLab CI pipeline. **GovernX** shifts the focus from correction to prevention : a three-stage CI/CD pipeline that provisions new projects from templates resolved by GitLab topic, aligns the project identity inside its build manifest, applies classification topics, and finally reuses the Audit-Tool rules to audit and fix the newly created project. The whole chain is *fail-closed* : any ambiguity stops the pipeline rather than producing a guessed configuration.

Keywords : DevOps, DevSecOps, GitLab, REST API, Python, CI/CD, governance, compliance, automation, Infrastructure as Code.

Table des matières

Dédicace	i
Remerciements	ii
Résumé / Abstract	iii
Table des matières	v
Table des figures et des tableaux	vii
Liste des abréviations	ix
Introduction générale	1
1 Contexte général du projet	3
1.1 Organisme d'accueil : CIH Bank	3
1.2 Établissement de formation : l'EMSI	3
1.3 Cadre du stage	4
1.4 Problématique	5
1.5 Objectifs et périmètre	5
1.6 Méthodologie et planification	6
2 Étude technique et choix technologiques	8
2.1 GitLab : modèle de données et concepts clés	8
2.2 L'API REST de GitLab	9
2.3 L'intégration continue GitLab	10
2.4 Choix technologiques	11
2.5 Positionnement par rapport à l'existant	12
3 Audit-Tool : audit et remédiation de conformité	13
3.1 Cahier des charges	13
3.2 Architecture	14
3.3 Implémentation des règles	15
3.4 Couche de remédiation automatique	17
3.5 Génération des rapports	19
3.6 Industrialisation en intégration continue	19
3.7 Tests et validation	20
4 GovernX : provisionnement de projets conformes par construction	22
4.1 Genèse et positionnement	22
4.2 Axe 1 : la chaîne de provisionnement	22
4.3 Décisions d'architecture	26
4.4 Stratégie de test	28
4.5 Perspectives : les axes 2 et 3	28

5	Bilan, difficultés et perspectives	30
5.1	Résultats obtenus	30
5.2	Analyse des bugs marquants	30
5.3	Considérations de sécurité	32
5.4	Limites connues	34
5.5	Perspectives d'évolution	34
5.6	Apports personnels	34
	Conclusion générale	36
A	Annexes	38
A.1	Audit-Tool — options de la ligne de commande	38
A.2	GovernX — variables de la chaîne CI/CD	38
A.3	Chaîne CI/CD et arborescence de GovernX	39
A.4	Glossaire	39
	Webographie et références	41

Table des figures

1.1	Positionnement du stage dans l'organisation de la DSI (vue simplifiée)	4
1.2	Planification prévisionnelle du stage (diagramme de Gantt simplifié)	7
2.1	Exécution séquentielle des stages et transmission d'état par artefacts	10
3.1	Architecture modulaire d'Audit-Tool	14
3.2	Sélection de la stratégie de correction selon l'édition de GitLab	18
4.1	Les trois étapes de la chaîne GOVERNX	23
5.1	Mécanisme du faux positif : une protection créée sur une branche inexistante	32

Liste des tableaux

1.1	Fiche signalétique du stage	4
1.2	Objectifs assignés au projet	6
2.1	Niveaux d'accès GitLab et valeurs numériques associées	9
2.2	Alternatives étudiées et raisons de leur écartement	12
3.1	Synthèse du référentiel de conformité implémenté	14
3.2	Scénarios de validation d'Audit-Tool	20
4.1	Ce que la copie miroir transporte réellement	24
4.2	Correspondance type de projet, technologie et manifeste	25
4.3	Décisions d'architecture structurantes	27
5.1	Atteinte des objectifs fixés au chapitre 1	30
5.2	Synthèse des cinq bugs marquants	33
5.3	Limites connues et assumées	34
A.1	Options de la ligne de commande d'Audit-Tool	38
A.2	Variables de déclenchement (formulaire « Run pipeline »)	38
A.3	Variables d'infrastructure (niveau groupe, masquées et protégées)	39
A.4	Glossaire des termes techniques	40

Liste des abréviations

Sigle	Signification
API	<i>Application Programming Interface</i> — interface de programmation applicative
CE	<i>Community Edition</i> — édition gratuite et libre de GitLab
CI/CD	<i>Continuous Integration / Continuous Delivery</i> — intégration et livraison continues
CIH	Crédit Immobilier et Hôtelier
CLI	<i>Command Line Interface</i> — interface en ligne de commande
CSV	<i>Comma-Separated Values</i> — format de fichier tabulaire
CVE	<i>Common Vulnerabilities and Exposures</i> — référentiel public de vulnérabilités
DSI	Direction des Systèmes d'Information
EE	<i>Enterprise Edition</i> — édition commerciale de GitLab
EMSI	École Marocaine des Sciences de l'Ingénieur
HTTP	<i>HyperText Transfer Protocol</i> — protocole de transfert web
IaC	<i>Infrastructure as Code</i> — infrastructure décrite sous forme de code
JSON	<i>JavaScript Object Notation</i> — format d'échange de données
MR	<i>Merge Request</i> — demande de fusion
PAT	<i>Personal Access Token</i> — jeton d'accès personnel GitLab
PFA	Projet de Fin d'Année
POM	<i>Project Object Model</i> — fichier <code>pom.xml</code> d'un projet Maven
REST	<i>Representational State Transfer</i> — style d'architecture d'API web
SCM	<i>Source Code Management</i> — gestion du code source
SI	Système d'Information
TLS	<i>Transport Layer Security</i> — protocole de chiffrement des échanges
URL	<i>Uniform Resource Locator</i> — adresse d'une ressource web
WSL	<i>Windows Subsystem for Linux</i>
XML	<i>eXtensible Markup Language</i> — format de balisage extensible
YAML	<i>YAML Ain't Markup Language</i> — format de configuration lisible

Introduction générale

Contexte

La transformation numérique du secteur bancaire marocain s’est traduite, ces dernières années, par une multiplication des applications développées en interne : applications mobiles, portails clients, interfaces d’échange avec les partenaires, traitements par lots nocturnes, briques de sécurité. Chacune de ces applications vit dans un dépôt de code, et l’ensemble de ces dépôts constitue aujourd’hui un patrimoine technique dont la valeur — et le risque — sont comparables à ceux des bases de données de production.

Cette réalité a une conséquence directe : le dépôt de code n’est plus un espace de travail privé laissé à la discrétion de l’équipe qui le remplit. Il devient un objet gouverné, soumis à des règles vérifiables. Qui a le droit de pousser directement sur la branche qui alimente la production ? Qui peut créer un tag de version, c’est-à-dire figer un artefact qui partira en livraison ? Une fusion peut-elle être acceptée sans que la chaîne d’intégration continue ait validé le code ? Ces questions relèvent de la conformité au même titre qu’un contrôle d’accès applicatif, et les référentiels normatifs auxquels une banque est soumise — ISO/CEI 27001 pour la sécurité de l’information, PCI-DSS pour les données de paiement — les rendent opposables lors d’un audit.

Problématique

CIH Bank héberge ses dépôts sur une instance GitLab auto-hébergée. Le référentiel de gouvernance existe : il a été défini par les équipes DevOps et documente ce qu’un dépôt conforme doit présenter. Le problème n’est donc pas l’absence de règles, mais l’absence de mécanisme pour les faire respecter à l’échelle. Trois symptômes ont motivé ce projet :

1. **Le contrôle est manuel et donc non reproductible.** Vérifier les protections de branches d’un projet demande d’ouvrir son écran de configuration, de lire trois écrans distincts et de comparer mentalement au référentiel. Répété sur plusieurs dizaines de projets, l’exercice n’est pas tenable, et son résultat n’est pas archivable en l’état.
2. **La dérive est silencieuse.** Une protection de branche supprimée « temporairement » pour débloquer une livraison ne se signale nulle part. Sans contrôle périodique, l’écart persiste jusqu’à ce qu’un incident le révèle.
3. **Chaque nouveau projet repart de zéro.** Lorsqu’une équipe crée un dépôt, elle reproduit — ou oublie de reproduire — une configuration attendue : structure de branches, protections, *topics* de classification, squelette applicatif. Le résultat dépend de la vigilance individuelle.

Objectifs du projet

Le travail réalisé durant ce stage répond à ces trois symptômes par deux livrables complémentaires, développés successivement.

Le premier, **Audit-Tool**, traite les symptômes 1 et 2 : automatiser la vérification du référentiel sur l’ensemble d’une instance, produire un rapport exploitable, et — au-delà du constat — corriger automatiquement les écarts détectés lorsque l’opérateur le demande explicitement.

Le second, **GovernX**, traite le symptôme 3 en déplaçant le curseur de la correction vers la prévention. Plutôt que d’auditer des projets qui ont déjà dérivé, il industrialise la création elle-même : un formulaire, une chaîne CI/CD en trois étapes, et un projet qui naît conforme parce que la même logique d’audit est exécutée automatiquement à sa création.

Fil directeur

Les deux projets ne sont pas juxtaposés : GOVERNX *réutilise* les règles métier d’Audit-Tool plutôt que de les réimplémenter. Ce principe de non-duplication de la logique métier est l’une des décisions d’architecture structurantes du travail, et il est discuté en détail au chapitre 4.

Organisation du rapport

Ce document est structuré en cinq chapitres.

Le **chapitre 1** présente le contexte général : l’organisme d’accueil, l’établissement de formation, le cadre du stage, la problématique détaillée et la méthodologie de travail adoptée.

Le **chapitre 2** pose les fondements techniques nécessaires à la compréhension des deux projets : le modèle de données de GitLab, son système de protections et de niveaux d’accès, le fonctionnement de son API REST et de sa chaîne CI/CD, ainsi que la justification des choix technologiques.

Le **chapitre 3** est consacré au premier livrable, Audit-Tool : cahier des charges, architecture, implémentation des cinq règles de conformité, couche de remédiation automatique, génération des rapports et industrialisation en CI.

Le **chapitre 4** traite du second livrable, GOVERNX : genèse, architecture en trois axes, description détaillée de la chaîne de provisionnement en trois étapes, décisions d’architecture et stratégie de test.

Le **chapitre 5** dresse le bilan : résultats obtenus, analyse détaillée des bugs les plus instructifs rencontrés durant le développement, considérations de sécurité, limites connues et perspectives d’évolution.

Une conclusion générale, des annexes techniques et une webographie complètent le document.

CHAPITRE 1

Contexte général du projet

Introduction du chapitre

Ce premier chapitre situe le travail dans son environnement. Il présente successivement l'organisme d'accueil et l'établissement de formation, précise le cadre et les conditions du stage, formalise la problématique métier qui a motivé le projet, énonce les objectifs retenus, et décrit enfin la méthodologie de travail ainsi que la planification suivie.

1.1 Organisme d'accueil : CIH Bank

1.1.1 Présentation générale

CIH Bank — de son nom historique *Crédit Immobilier et Hôtelier* — est un établissement bancaire marocain dont le siège se situe à Casablanca. Créé dans la première moitié du XX^e siècle avec une vocation initiale de financement de l'immobilier et de l'hôtellerie, l'établissement a progressivement élargi son périmètre pour devenir une banque universelle, adressant aujourd'hui aussi bien la clientèle des particuliers que celle des entreprises et des professionnels. La banque fait partie du groupe *Caisse de Dépôt et de Gestion* (CDG) et est cotée à la Bourse de Casablanca.

CIH Bank s'est distinguée sur le marché marocain par un positionnement fortement orienté vers le digital : offre bancaire à distance, ouverture de compte dématérialisée, applications mobiles et services en ligne constituent une part importante de sa relation client. Ce positionnement a une conséquence directe sur l'organisation interne : la Direction des Systèmes d'Information n'y est pas une fonction support périphérique, mais un acteur central de la production de valeur.

Les éléments chiffrés (effectifs, nombre d'agences, résultats financiers) n'ont volontairement pas été repris ici : ils évoluent d'un exercice à l'autre et relèvent de la communication institutionnelle de la banque.

1.1.2 La Direction des Systèmes d'Information

Le stage s'est déroulé au sein de la Direction des Systèmes d'Information (DSI), et plus précisément dans l'équipe en charge des pratiques DevOps et de la chaîne d'outillage de développement. Cette équipe a la responsabilité transverse de la plateforme sur laquelle toutes les équipes applicatives de la banque produisent et livrent leur code.

Ses missions couvrent notamment :

- l'administration de la plateforme de gestion de code source (GitLab auto-hébergée) et des exécuteurs d'intégration continue associés ;
- la définition et la diffusion des standards techniques : conventions de branches, stratégie de versionnement, structure des chaînes CI/CD ;
- l'accompagnement des équipes applicatives dans l'adoption de ces standards ;
- la sécurisation de la chaîne de livraison logicielle, en lien avec les exigences de conformité auxquelles un établissement bancaire est soumis.

C'est cette dernière mission qui a directement donné naissance au sujet de stage.

1.2 Établissement de formation : l'EMSI

L'École Marocaine des Sciences de l'Ingénieur (EMSI) est un établissement d'enseignement supérieur privé marocain, fondé en 1986 et implanté sur plusieurs campus dont Casablanca, Rabat, Marrakech, Tanger et Fès. Elle est membre du réseau *Honoris United Universities*, premier réseau

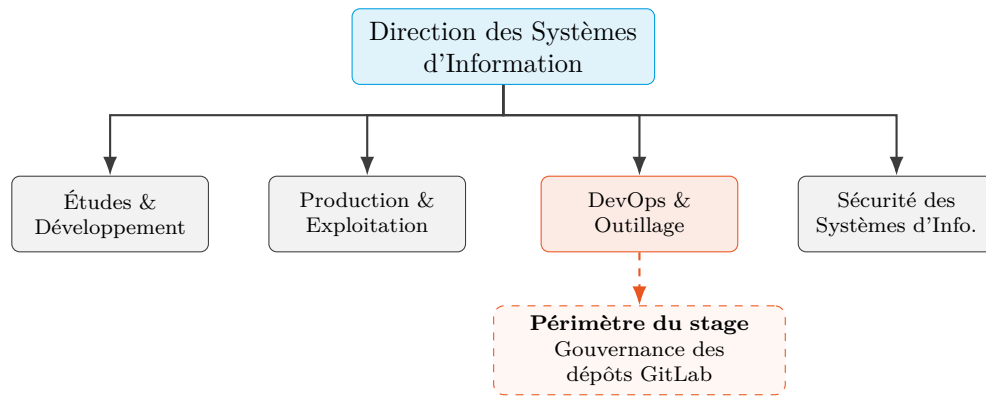


Figure 1.1 – Positionnement du stage dans l’organisation de la DSI (vue simplifiée)

d’enseignement supérieur privé panafricain. L’école forme des ingénieurs en cinq ans, avec un cycle préparatoire intégré de deux ans suivi d’un cycle ingénieur de trois ans réparti en plusieurs filières : Génie Informatique et Réseaux, Génie Civil, Génie Industriel, Génie Financier et Génie Électrique.

La filière **Génie Informatique et Réseaux**, dans laquelle s’inscrit ce travail, couvre l’algorithmique et le génie logiciel, les bases de données, les réseaux et les systèmes d’exploitation, la sécurité informatique, ainsi que les architectures distribuées et le *cloud computing*. Le cursus impose à chaque niveau la réalisation d’un projet en entreprise, dont le Projet de Fin d’Année (PFA) en troisième année du cycle ingénieur, objet du présent rapport.

1.3 Cadre du stage

Table 1.1 – Fiche signalétique du stage

Intitulé	Automatisation de la gouvernance et de la conformité des dépôts GitLab
Nature	Projet de Fin d’Année (PFA) — 3 ^{ème} année cycle ingénieur
Étudiant	Chouaib MOUTAOUAKIL
Filière	Génie Informatique et Réseaux — EMSI Casablanca
Organisme d’accueil	CIH Bank — Direction des Systèmes d’Information
Encadrant professionnel	M. Yassine NAJI
Période	Juillet – août 2026
Livrables	Audit-Tool : outil CLI d’audit et de remédiation GovernX : chaîne CI/CD de provisionnement conforme
Technologies	Python 3.11, <i>python-gitlab</i> , API REST GitLab v4, GitLab CI, Docker, Git

1.3.1 Environnement de travail

Le développement n’a pas été mené directement sur l’instance GitLab de production. Un environnement de développement isolé a été mis en place, ce qui constituait en soi un premier exercice technique :

- une instance **GitLab Community Edition** déployée en conteneur Docker, hébergée sur une distribution Ubuntu exécutée dans WSL2 ;
- un *runner* GitLab enregistré sur cette instance et configuré avec l’exécuteur Docker, permettant l’exécution réelle des chaînes CI/CD développées ;
- plusieurs dépôts de test créés ou importés depuis des projets publics (un projet Maven, un projet npm, un dépôt multi-branches) afin de disposer de cas réels et variés plutôt que de fichiers minimaux artificiels ;
- une instance GitLab.com utilisée comme hébergeur des dépôts modèles pour le second projet, ce qui a introduit une contrainte d’architecture *cross-instance* détaillée au chapitre 4.

La mise en place de cet environnement a mobilisé les premiers jours du stage et a donné lieu à ses propres difficultés : configuration de l'`external_url` de GitLab, résolution réseau entre WSL2 et l'hôte Windows, enregistrement du *runner*. Ces obstacles, sans rapport direct avec le sujet, ont constitué une première familiarisation utile avec la réalité d'une plateforme auto-hébergée.

Il est par ailleurs honnête de préciser que le stage a débuté sans expérience préalable du langage Python ni de l'API GitLab. L'apprentissage s'est fait au fil de l'eau, directement sur le code du projet. Ce choix — apprendre en produisant plutôt qu'apprendre puis produire — a eu un coût : plusieurs bugs décrits au chapitre 5 sont imputables à une compréhension initialement incomplète, soit du langage, soit du modèle de données de GitLab. Il a aussi eu le bénéfice d'ancrer chaque notion dans un usage concret.

1.4 Problématique

1.4.1 Formulation

La problématique peut se formuler en une phrase :

Problématique

Comment garantir, de manière automatisée, vérifiable et reproductible, que l'ensemble des dépôts de code d'une instance GitLab respecte le référentiel de gouvernance défini par l'établissement — et comment faire en sorte qu'un nouveau dépôt soit conforme dès sa création plutôt que corrigé après coup ?

1.4.2 Décomposition en sous-problèmes

Cette question se décompose en quatre sous-problèmes techniques distincts.

L'inventaire. Énumérer de façon fiable tous les projets d'une instance, y compris ceux répartis dans des sous-groupes imbriqués, tout en excluant un périmètre donné. Cette question, apparemment triviale, se heurte à la pagination de l'API et à la distinction entre projets appartenant à un groupe et projets simplement visibles.

L'expression des règles. Une règle formulée en langage naturel (« seuls les mainteneurs peuvent pousser sur la branche principale ») doit être traduite en une comparaison exécutable sur la représentation que l'API renvoie. Cette traduction n'est pas mécanique : elle suppose de comprendre le modèle d'accès de GitLab, qui repose sur des seuils hiérarchiques et non sur des listes de rôles. Une erreur de modélisation à ce niveau produit des faux positifs ou des faux négatifs systématiques.

La correction. Détecter un écart a peu de valeur si la correction reste manuelle. Or l'API GitLab n'offre pas systématiquement de mécanisme de modification en place : certaines ressources doivent être supprimées puis recrées, et certains champs ne sont accessibles que sur les éditions commerciales du produit.

La prévention. Corriger en boucle des projets qui dérivent est un travail sans fin. Il faut agir en amont, au moment de la création du projet, en industrialisant celle-ci à partir de dépôts modèles validés.

1.5 Objectifs et périmètre

Trois sujets ont été explicitement écartés, non par manque d'intérêt mais pour préserver la cohérence du livrable dans le temps imparti : l'**interface graphique** — un portail développeur a été étudié et planifié mais non réalisé, et figure en perspective au chapitre 4 ; le **déploiement en production**, qui relève d'une décision de la DSI et suppose des arbitrages sur les droits du compte de service ; et la **migration vers le cloud public**, dont la sensibilité pour les données de gouvernance d'un établissement bancaire impose des validations sécurité dépassant le cadre d'un PFA.

Table 1.2 – Objectifs assignés au projet

Réf.	Objectif	Critère de réussite
O1	Auditer l'ensemble d'une instance GitLab	Toutes les règles évaluées sur tous les projets, hors périmètre exclu, sans intervention manuelle
O2	Produire un rapport exploitable	CSV (tableur), JSON (outil tiers), HTML (consultation) ; historisation des exécutions
O3	Corriger automatiquement les écarts	Remédiation optionnelle, avec mode simulation obligatoire avant toute écriture réelle
O4	Industrialiser l'exécution	Chaîne GitLab CI déclenchable manuellement ou par planification, avec archivage des rapports
O5	Provisionner des projets conformes	Création depuis un dépôt modèle, configuration et audit automatiques, en une seule exécution
O6	Garantir l'absence de duplication	La logique métier des règles n'existe qu'en un seul endroit, réutilisée par les deux projets

1.6 Méthodologie et planification

1.6.1 Approche adoptée

Le travail a suivi une démarche itérative et incrémentale, structurée par des points réguliers avec l'encadrant professionnel. Chaque itération suivait le même schéma : **cadrage** d'un point du référentiel ou d'un choix d'architecture avec l'encadrant, formalisé sous forme de « décision actée » ; **implémentation** avec écriture des tests unitaires associés ; **validation locale** contre l'instance de développement, sur des dépôts de test représentatifs ; **validation en CI**, seul environnement représentatif de l'usage réel ; et enfin **réconciliation documentaire** du document de suivi.

1.6.2 Traçabilité des décisions

Un choix méthodologique mérite d'être souligné car il a eu un impact réel sur la qualité du travail : la tenue d'un **document de décisions daté et versionné**. Chaque arbitrage d'architecture y est consigné avec sa date, sa justification, et le cas échéant la mention explicite qu'il en supersède un précédent.

Ce dispositif a permis, à plusieurs reprises, de détecter des *écarts entre ce que la documentation affirmait et ce que le code contenait réellement*. Une correction décrite comme appliquée pouvait, en réalité, n'avoir jamais été commise. La leçon retenue — vérifier l'état réel des fichiers avant d'empiler des corrections — est développée au chapitre 5.

1.6.3 Planification

Conclusion du chapitre

Ce chapitre a posé le décor : une banque dont le patrimoine applicatif croît, une direction informatique qui a défini un référentiel de gouvernance mais ne dispose pas des moyens de le faire respecter à l'échelle, et un stage dont l'objectif est de combler cet écart par deux outils complémentaires. Le chapitre suivant établit les fondements techniques nécessaires : que sont exactement une branche protégée, un tag protégé et un *topic* GitLab, comment l'API les expose, et pourquoi ces détails apparemment secondaires conditionnent l'ensemble de la conception.

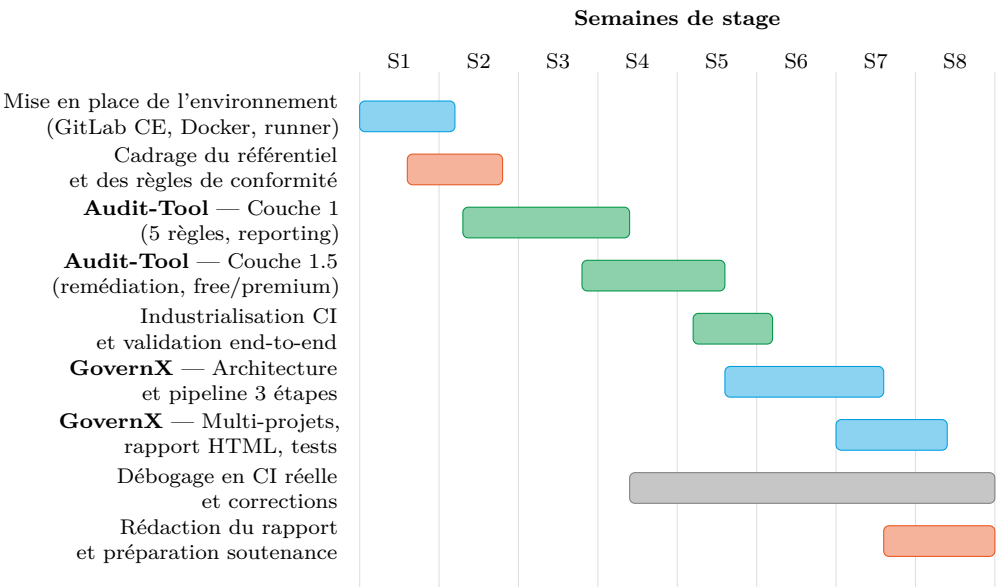


Figure 1.2 – Planification prévisionnelle du stage (diagramme de Gantt simplifié)

CHAPITRE 2

Étude technique et choix technologiques

Introduction du chapitre

Les deux outils développés durant ce stage manipulent en permanence les mêmes objets : projets, groupes, branches protégées, tags protégés, *topics*, niveaux d'accès. Une part importante des difficultés rencontrées ne vient pas de la programmation elle-même, mais d'une compréhension initialement approximative de ces objets et de la façon dont l'API les représente. Ce chapitre établit donc les fondements indispensables avant d'aborder l'implémentation, puis justifie les choix technologiques retenus.

2.1 GitLab : modèle de données et concepts clés

2.1.1 Groupes, sous-groupes et projets

GitLab organise ses ressources dans une arborescence de **namespaces**. Un projet appartient soit à un utilisateur, soit à un groupe, et un groupe peut contenir des sous-groupes sur une profondeur arbitraire. Chaque projet est identifié de deux manières : par un **identifiant numérique** unique et stable, et par son **chemin complet** (`path_with_namespace`), par exemple `cih-internship/mon-projet`, lisible mais susceptible de changer si le projet est renommé ou déplacé. Cette dualité a une conséquence pratique immédiate : lorsqu'un chemin est utilisé dans une URL d'API, il doit être encodé (les / deviennent %2F). Le client REST développé pour GOVERNIX encapsule systématiquement cet encodage plutôt que de le laisser à la charge de chaque appel.

2.1.2 Branches protégées

Une **branche protégée** est une règle stockée en base de données GitLab, associée à un projet, qui restreint deux opérations sur une branche ou un motif de branches :

- `push_access_levels` — qui peut pousser directement des commits ;
- `merge_access_levels` — qui peut fusionner une demande de fusion vers cette branche.

Le nom de la règle peut être un nom exact (`main`) ou un motif avec joker (`release/*`), auquel cas la règle s'applique à toutes les branches correspondantes.

Point critique : le seuil, pas la liste

Le point le plus important — et la source du bug le plus coûteux du projet, analysé au chapitre 5 — est que **GitLab modélise un niveau d'accès comme un seuil minimal, et non comme une liste de rôles autorisés**. Déclarer `merge_access_level = 30` (Developer) signifie « Developer *et tout rôle supérieur* », soit Developer, Maintainer et Owner. Il ne faut donc pas envoyer deux entrées `{30, 40}` pour exprimer « Developers et Maintainers » : une seule entrée `{30}` suffit, et GitLab ne persistera de toute façon pas la seconde.

2.1.3 Tags protégés

Un **tag protégé** suit une logique voisine, avec une différence fondamentale : il ne comporte qu'une seule dimension, `create_access_levels`, qui définit qui a le droit de créer un tag correspondant au motif. La protection des tags est essentielle dans un contexte de livraison : un tag matérialise une version figée, et sa création déclenche généralement une chaîne de publication d'artefact.

Une seconde différence, découverte durant le développement, est que **l'API des tags protégés n'a jamais exposé de méthode de modification en place**, quelle que soit l'édition de GitLab.

Table 2.1 – Niveaux d'accès GitLab et valeurs numériques associées

Valeur	Rôle	Portée
0	No access	Personne (utilisé pour verrouiller totalement une action)
5	Minimal	Accès minimal, rarement utilisé dans un contexte de protection
10	Guest	Lecture des tickets, pas d'accès au code sur un projet privé
20	Reporter	Lecture du code, pas d'écriture
30	Developer	Écriture sur les branches non protégées, création de MR
40	Maintainer	Configuration du projet, gestion des protections
50	Owner	Contrôle total sur le projet ou le groupe
60	Admin	Administrateur de l'instance — palier accepté par certains points d'entrée, dont celui des tags protégés

Seules les opérations de lecture, de création et de suppression existent. Toute correction d'un niveau d'accès passe donc obligatoirement par une séquence *supprimer puis recréer*.

2.1.4 Topics

Les **topics** sont des étiquettes libres attachées à un projet. Purement descriptifs à l'origine — ils servent à la recherche et au classement dans l'interface — ils sont utilisés dans GOVERN_X comme mécanisme de résolution : c'est le *topic* porté par un dépôt modèle qui permet de le retrouver à partir du type de projet demandé par l'utilisateur, sans avoir à maintenir un tableau de correspondance en dur dans le code.

2.1.5 Le paramètre « Pipeline must succeed »

Il s'agit d'un simple attribut booléen du projet, `only_allow_merge_if_pipeline_succeeds`. Lorsqu'il est actif, GitLab refuse la fusion d'une demande tant que la dernière chaîne d'intégration associée n'est pas passée au vert. C'est la règle la plus simple des cinq à vérifier comme à corriger, et elle sert de cas de référence dans les tests.

2.2 L'API REST de GitLab

2.2.1 Principes

GitLab expose une API REST en version 4, accessible sous `/api/v4/`. L'authentification se fait par jeton, transmis dans l'en-tête HTTP `PRIVATE-TOKEN`. Les jetons sont porteurs de *scopes* : le scope `read_api` suffit pour un audit en lecture seule, mais le scope `api` — lecture et écriture — devient nécessaire dès lors que l'outil doit corriger les écarts qu'il détecte.

Le scope ne suffit pas

Un point de vigilance identifié durant le stage et important pour une future mise en production : **le scope du jeton ne détermine pas à lui seul ce que l'outil peut faire**. Le compte propriétaire du jeton doit également disposer du rôle adéquat sur les projets concernés. Lister les branches protégées d'un projet exige le rôle *Maintainer* ; un jeton de scope `api` appartenant à un compte simplement *Developer* recevra un **403 Forbidden**. Cette distinction a coûté plusieurs heures de débogage.

2.2.2 Pagination

L'API pagine ses réponses, par défaut à 20 éléments. Ignorer ce comportement conduit à des résultats silencieusement tronqués — un audit qui déclare « conforme » un projet dont les fichiers révélateurs se trouvaient au-delà de la vingtième entrée. Deux stratégies ont été employées selon la bibliothèque :

- avec *python-gitlab*, les paramètres `get_all=True` ou `iterator=True` délèguent la pagination à la bibliothèque ;

- avec le client REST développé pour GOVERNIX, une boucle explicite suit l'en-tête de réponse `X-Next-Page` jusqu'à épuisement.

2.2.3 Éditions Community et Enterprise

GitLab est distribué en édition *Community* (CE, libre et gratuite) et en éditions commerciales *Premium* et *Ultimate*. Certains points d'entrée d'API se comportent différemment selon l'édition. Le cas rencontré durant le stage est particulièrement instructif :

- le point d'entrée `PATCH /projects/:id/protected_branches/:name` n'accepte les champs `allowed_to_push` et `allowed_to_merge` — les seuls permettant de modifier les niveaux d'accès — que sur les éditions *Premium* et *Ultimate* ;
- sur l'édition *Community*, ces champs sont **silencieusement ignorés** : le serveur répond 200 OK sans rien modifier.

Ce « faux succès » a été la source d'un bug particulièrement déroutant, analysé au chapitre 5, et a conduit à une décision d'architecture : implémenter deux stratégies de correction distinctes, sélectionnées par un paramètre `-tier`.

2.3 L'intégration continue GitLab

2.3.1 Modèle d'exécution

Une chaîne GitLab CI est décrite dans un fichier `.gitlab-ci.yml` placé à la racine du dépôt. Elle s'organise en **stages** exécutés séquentiellement, chaque stage contenant un ou plusieurs **jobs** exécutés en parallèle. Les jobs sont pris en charge par des **runners**, ici configurés avec l'exécuteur Docker : chaque job démarre dans un conteneur neuf, ce qui garantit l'isolation mais implique que rien ne persiste d'un job à l'autre sans mécanisme explicite.

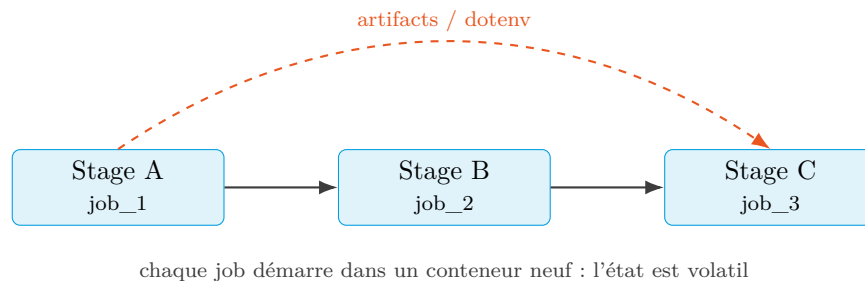


Figure 2.1 – Exécution séquentielle des stages et transmission d'état par artefacts

2.3.2 Transmission d'état entre jobs

Puisque chaque job repart d'un conteneur vierge, la transmission d'information entre étapes passe par les **artefacts**. Deux formes sont utilisées dans GOVERNIX :

- les **artefacts fichiers**, archivés à la fin d'un job et téléchargeables ou réutilisés par les jobs suivants (les rapports d'audit notamment) ;
- les **artefacts dotenv**, un fichier de paires `CLÉ=valeur` dont GitLab injecte le contenu comme variables d'environnement des jobs suivants. C'est le mécanisme employé pour transmettre la liste des projets créés par la première étape aux deux étapes suivantes.

2.3.3 Variables et déclenchement

Les variables peuvent être définies à plusieurs niveaux (instance, groupe, projet, exécution). Deux attributs sont essentiels pour la sécurité : **masquée**, qui occulte la valeur dans les journaux d'exécution, et **protégée**, qui n'expose la variable qu'aux jobs s'exécutant sur une branche ou un tag

protégé. Les jetons d'API utilisés par les deux projets sont systématiquement déclarés masqués et protégés, au niveau du groupe racine plutôt que projet par projet.

Enfin, le bloc `workflow:rules` permet de conditionner la création même d'une chaîne. GOVERNIX l'utilise pour n'autoriser que le déclenchement manuel via le formulaire « *Run pipeline* », par la règle `$CI_PIPELINE_SOURCE == "web"`. Ce choix est délibéré : il empêche qu'un simple `git push` déclenche involontairement la création de projets, et il est plus strict qu'un `when: manual` appliqué job par job, qui laisserait la chaîne se créer avant d'attendre une validation.

2.4 Choix technologiques

2.4.1 Langage : Python

Python a été retenu pour l'ensemble du code des deux projets, pour quatre raisons : une **bibliothèque cliente mature**, *python-gitlab*, qui couvre l'ensemble de l'API v4 avec un modèle objet cohérent ; l'**homogénéité** — écrire les scripts de la chaîne CI en Bash et la logique métier en Python aurait créé deux univers de test distincts, d'où le choix « tout en Python, pas de Bash », acté explicitement ; la **testabilité**, *pytest* permettant de tester la logique métier contre des objets simulés sans instance GitLab ni accès réseau ; et enfin l'**accessibilité**, point non négligeable au vu du niveau de départ.

2.4.2 Bibliothèques et dépendances

Quatre dépendances suffisent à l'ensemble : *python-gitlab* (8.4.0), client objet de l'API GitLab couvrant projets, branches protégées, tags protégés et membres ; *requests* (≥ 2.31), client HTTP bas niveau utilisé par le wrapper REST de GOVERNIX pour les appels non couverts ; *python-dotenv* (1.2.2), pour le chargement de la configuration locale ; et *pytest* (≥ 8.0) comme cadre de test.

Pourquoi deux clients ?

Une question légitime est de savoir pourquoi GOVERNIX n'utilise pas exclusivement *python-gitlab*. La réponse tient à la séparation des responsabilités : les étapes d'*orchestration* (création de projet, écriture d'un commit, lecture d'une arborescence) utilisent un wrapper REST maison, léger et entièrement maîtrisé, tandis que les *règles d'audit* réutilisent le modèle objet de *python-gitlab*, exactement comme dans Audit-Tool. Cette symétrie est ce qui rend possible la réutilisation des règles sans réécriture.

2.4.3 Choix de conception structurants

Quatre choix ont été arbitrés en début de projet et n'ont pas été remis en cause.

Fonctions plutôt que classes : avec quatre à cinq règles fixes et une logique sans état, l'orientation objet aurait introduit une indirection sans bénéfice ; chaque règle est une paire `check_*` / `fix_*`.

Convention de retour uniforme : toutes les fonctions de vérification renvoient un dictionnaire portant les mêmes clés (`rule`, `project`, `compliant`, `details`), éventuellement enrichi de clés spécifiques. Le générateur de rapports traite ainsi l'ensemble des résultats dans une seule boucle générique, ce qui rend l'ajout d'une sixième règle quasiment gratuit.

Comportement *fail-closed* : en cas d'ambiguïté — manifeste introuvable, plusieurs dépôts modèles candidats, branche par défaut indéterminée — l'outil échoue explicitement plutôt que de deviner. Une protection appliquée au mauvais motif est pire qu'une absence de protection signalée.

Aucun mécanisme de repli silencieux : un repli qui masque l'absence de logique métier réelle donne l'illusion du fonctionnement. Cette règle a conduit à supprimer plusieurs *fallbacks* initialement présents.

2.5 Positionnement par rapport à l'existant

Plusieurs approches concurrentes existent pour traiter la conformité d'une plateforme SCM. Elles ont été écartées pour les raisons suivantes.

Table 2.2 – Alternatives étudiées et raisons de leur écartement

Approche	Principe	Raison de l'écartement
Terraform <i>provider</i> GitLab	Décrire les protections en IaC et appliquer l'état désiré	Suppose que tous les projets soient déjà sous gestion Terraform ; ne répond pas au besoin d'auditer un parc existant non géré
Politiques natives GitLab	Utiliser les <i>compliance frameworks</i>	Fonctionnalité réservée aux éditions Ultimate ; instance de développement en édition Community
Scripts ponctuels	Un script par règle, exécuté à la demande	Aucun rapport consolidé, aucune historisation, aucune correction : c'est la situation de départ

La solution retenue est donc un outil unique à règles centralisées, produisant un rapport consolidé, offrant une correction optionnelle, et réutilisable par la chaîne de provisionnement.

Conclusion du chapitre

Les concepts posés ici — seuil d'accès plutôt que liste de rôles, absence de modification en place sur les tags protégés, divergence de comportement entre éditions, pagination, transmission d'état par artefacts — ne sont pas des détails d'implémentation : ce sont les contraintes qui ont façonné l'architecture des deux outils. Le chapitre suivant présente le premier d'entre eux.

CHAPITRE 3

Audit-Tool : audit et remédiation de conformité

Introduction du chapitre

Ce chapitre présente le premier livrable du stage. Il détaille le cahier des charges négocié avec l'encadrant, l'architecture retenue, l'implémentation des cinq règles de conformité, la couche de remédiation automatique, la génération des rapports, et enfin l'industrialisation de l'ensemble dans une chaîne d'intégration continue.

3.1 Cahier des charges

3.1.1 Expression du besoin

Le besoin initial était formulé simplement : *parcourir tous les dépôts d'une instance GitLab auto-hébergée, à l'exception d'un groupe donné, et vérifier qu'ils respectent les règles de conformité définies par l'équipe DevOps*. Le délai fixé pour une première démonstration était d'une semaine.

Ce cadrage initial a été affiné par une série de questions posées à l'encadrant, dont les réponses ont structuré la conception : la règle d'accès utilisateur doit *agir* et non seulement auditer (ajout ou ajustement du membre) ; le rôle minimum pour fusionner sur `release/*` est *Developer* ; la règle s'applique à tous les projets audités, avec un identifiant fourni une seule fois pour toute l'exécution ; chaque action doit apparaître dans le rapport. Une seule question est restée ouverte, celle des droits du compte de service en production, à valider avec l'équipe infrastructure. Ces échanges illustrent qu'une spécification technique se construit par itérations plutôt qu'elle ne se reçoit d'un bloc.

3.1.2 Les cinq règles de conformité

Règle 1 — Branches protégées (générique). Les branches structurantes du modèle de branches interne doivent être protégées avec les niveaux d'accès attendus. Trois motifs sont contrôlés : la branche principale, `develop`, et le motif `release/*`.

Règle 2 — Branches `release/*` (spécifique). Ce cas particulier a été explicité par l'encadrant : la fusion doit être ouverte à partir du rôle *Developer*, tandis que la poussée directe doit rester réservée au rôle *Maintainer*. C'est cette asymétrie qui rend la règle intéressante — et qui a révélé le bug de modélisation analysé au chapitre 5.

Règle 3 — Tags protégés avec détection dynamique. Un projet doit protéger ses tags de version. Le motif attendu n'est cependant pas identique pour tous les projets : il dépend de la technologie. Un projet Maven, identifié par la présence d'un `pom.xml`, utilise une convention différente d'un projet npm, identifié par un `package.json`. La règle détecte donc le *stack* avant de déterminer le motif à vérifier.

Règle 4 — « Pipeline must succeed ». L'attribut `only_allow_merge_if_pipeline_succeeds` doit être activé, afin qu'aucune fusion ne puisse intervenir sans validation de la chaîne d'intégration.

Règle 5 — Accès d'un utilisateur de service (facultative). Un identifiant d'utilisateur peut être fourni au lancement. S'il l'est, l'outil s'assure que cet utilisateur dispose d'un accès suffisant sur chaque projet audité. S'il ne l'est pas, la règle est simplement ignorée. C'est historiquement la seule règle qui écrit dans GitLab dès la phase de vérification, les quatre autres étant en lecture seule.

Quatre contraintes non fonctionnelles complètent ce périmètre : **trois formats de sortie** (CSV pour le tableur, JSON pour un outil tiers, HTML pour la consultation) ; un **périmètre paramétrable** par exclusion d'un groupe ; la **robustesse**, une erreur d'API sur un projet ne devant jamais interrompre l'audit des suivants ; et la **simulation obligatoire**, toute correction devant

Table 3.1 – Synthèse du référentiel de conformité implémenté

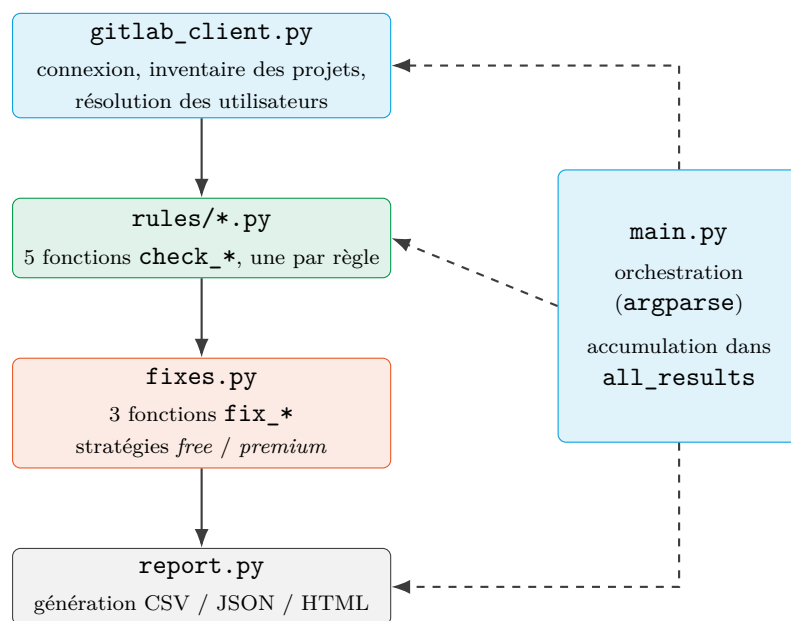
Règle	Cible	Attendu
protected_branches	main	<i>push</i> : Maintainer (40) — <i>merge</i> : Maintainer (40)
protected_branches	develop	<i>push</i> : Maintainer (40) — <i>merge</i> : Developer (30)
protected_branches	release/*	<i>push</i> : Maintainer (40) — <i>merge</i> : Developer (30)
protected_tags	motif dynamique	<i>create</i> : Maintainer (40) ; motif * si Maven, v* si npm
pipeline_must_succeed	projet	only_allow_merge_if_pipeline_succeeds = true
specific_user_access	membre	rôle Developer (30) minimum, ou accès nommé sur branche en édition Premium

pouvoir être simulée avant d’être appliquée.

3.2 Architecture

3.2.1 Vue d’ensemble

L’outil est structuré en quatre responsabilités clairement séparées : la connexion et l’inventaire, les règles, la remédiation, et le reporting. Un module d’orchestration les assemble.

**Figure 3.1** – Architecture modulaire d’Audit-Tool

Le paquet `audit_tool` regroupe ainsi `gitlab_client.py` (connexion, inventaire, résolution des identifiants d’utilisateurs), un sous-paquet `rules/` contenant un module par règle, `fixes.py` centralisant la remédiation, `report.py` pour les trois formats de sortie, et `main.py` pour l’orchestration. Deux fichiers complètent l’ensemble à la racine : `pyproject.toml` pour le packaging et `.gitlab-ci.yml` pour l’industrialisation.

3.2.2 Conventions de conception

Deux conventions gouvernent tout le code et méritent d’être explicitées, car ce sont elles qui permettent au reste de rester simple.

Convention de retour des vérifications. Toute fonction `check_*` renvoie un dictionnaire portant au minimum quatre clés communes. Des clés supplémentaires, spécifiques à la règle (`branch`, `pattern`, `project_type`, `user_id`), peuvent s’y ajouter.

```

1 {
2     "rule": "protected_branches",      # nom de la règle
3     "project": "cih-internship/api",  # path_with_namespace

```

```

4     "compliant": False,                # verdict
5     "details": "Règle absente : la branche n'est pas protégée",
6     # + clés optionnelles spécifiques : branch, pattern, project_type...
7 }

```

Extrait 3.1 – Convention de retour des fonctions de vérification

L'intérêt de cette uniformité est direct : le générateur de rapports parcourt une liste plate de dictionnaires avec une seule boucle générique, sans connaître les règles. Ajouter une sixième règle ne demande aucune modification de `report.py`.

Convention de retour des corrections. Symétriquement, les fonctions `fix_*` renvoient une structure miroir, dans laquelle `compliant` est remplacé par `success`, et où `action` décrit ce qui a été fait — ou ce qui aurait été fait en mode simulation — accompagné d'un indicateur `dry_run`. L'absence de la clé `compliant` sert de discriminant : elle permet à l'affichage console et au rapport HTML de distinguer une ligne de vérification d'une ligne de correction sans champ de type explicite.

3.3 Implémentation des règles

3.3.1 Connexion et inventaire

Le module `gitlab_client.py` expose trois fonctions. La connexion authentifie le jeton immédiatement (`gl.auth()`) plutôt que d'attendre le premier appel métier : un jeton invalide échoue ainsi au démarrage, avec un message explicite, et non au milieu d'un parcours de projets.

L'inventaire s'appuie sur un itérateur, qui délègue la pagination à la bibliothèque, et applique le filtre d'exclusion par préfixe de chemin. Un détail mérite l'attention : le préfixe est comparé suivi d'une barre oblique (`EXCLUDED_GROUP_PATH + "/"`). Sans elle, un groupe nommé `sandbox` exclurait également un groupe `sandbox-prod`, qui n'a rien à voir avec lui.

La troisième fonction, `resolve_user_ids`, accepte une liste d'identifiants séparés par des virgules, chacun pouvant être un identifiant numérique ou un nom d'utilisateur. Elle a été introduite après un retour d'utilisation : imposer un identifiant numérique obligeait l'opérateur à le chercher dans l'interface avant chaque exécution. Un identifiant non résolu émet un avertissement mais n'interrompt pas la résolution des autres.

3.3.2 Règle des branches protégées

L'implémentation compare, pour chaque motif attendu, l'ensemble des niveaux d'accès effectivement configurés à l'ensemble attendu :

```

1  MAINTAINER, DEVELOPER = 40, 30
2
3  EXPECTED_RULES = [
4      {"pattern": "main",      "push": {MAINTAINER}, "merge": {MAINTAINER}},
5      {"pattern": "develop",   "push": {MAINTAINER}, "merge": {DEVELOPER}},
6      {"pattern": "release/*", "push": {MAINTAINER}, "merge": {DEVELOPER}},
7  ]
8
9  def check_protected_branches(project):
10     protected_by_name = {pb.name: pb
11                          for pb in project.protectedbranches.list(get_all=True)}
12     results = []
13
14     for rule in EXPECTED_RULES:
15         pb = protected_by_name.get(rule["pattern"])
16         if pb is None:
17             results.append({... , "compliant": False,
18                             "details": "Règle absente : la branche n'est pas protégée"})
19             continue
20
21         actual_push = {lvl["access_level"] for lvl in pb.push_access_levels}
22         actual_merge = {lvl["access_level"] for lvl in pb.merge_access_levels}
23         push_ok, merge_ok = (actual_push == rule["push"],
24                              actual_merge == rule["merge"])
25         ...

```

Extrait 3.2 – Cœur de la règle des branches protégées

Le point notable est que chaque valeur attendue est un ensemble à *un seul élément*. Cette forme est la traduction directe du modèle de seuil décrit au chapitre 2 : « Developers et Maintainers peuvent fusionner » ne s'écrit pas `{30, 40}` mais `{30}`, car le rôle `Maintainer` hérite automatiquement de ce que le rôle `Developer` autorise. La première version du code faisait l'erreur inverse et produisait un échec systématique après correction ; l'analyse complète figure au chapitre 5.

Cette règle retourne une *liste* de résultats — un par motif — là où les autres n'en retournent qu'un seul. L'orchestrateur utilise donc `.extend()` pour celle-ci et `.append()` pour les autres.

3.3.3 Règle des tags protégés et détection du *stack*

Cette règle est la plus riche des cinq, car le motif attendu n'est pas connu à l'avance : il se déduit de la technologie du projet.

```

1 def detect_version_pattern(project):
2     default_branch = project.default_branch or "main"
3
4     try:
5         tree = project.repository_tree(ref=default_branch,
6                                       recursive=True, get_all=True)
7     except Exception as e:
8         print(f"[DEBUG] Impossible de lister l'arbre du repo : {e}")
9         return None, None
10
11     filenames = {item["path"].split("/")[-1]
12                  for item in tree if item["type"] == "blob"}
13
14     if "pom.xml" in filenames:
15         return "*", "maven"
16     if "package.json" in filenames:
17         return "v*", "npm"
18     return None, None

```

Extrait 3.3 – Détection dynamique du stack par le manifeste de build

Un bug révélateur : la recherche à la racine seulement

La première version utilisait `project.files.get(file_path="pom.xml", ref=branch)`, qui ne teste que la racine exacte du dépôt. Le résultat était incohérent : un projet Maven de test importé depuis un dépôt public, dont le `pom.xml` se trouve dans un sous-dossier `my-app/`, était déclaré « aucun manifeste détecté » alors que le fichier était visible dans l'interface. La correction consiste à parcourir *toute* l'arborescence et à comparer les noms de fichiers, et non les chemins. Le paramètre `get_all=True` est indispensable : sans lui, la pagination par défaut à vingt entrées tronque silencieusement l'arbre sur les dépôts volumineux. Ce bug est important au-delà de sa correction, car il illustre un piège récurrent : *une vérification qui ne trouve rien n'est pas la preuve qu'il n'y a rien*.

Le comportement en l'absence de manifeste est délibéré : la règle retourne `compliant: False` avec un message explicite plutôt que de deviner un motif. La correction associée refuse alors symétriquement d'agir. Un motif de protection erroné protégerait les mauvais tags tout en donnant l'illusion de la conformité.

3.3.4 Règle « Pipeline must succeed »

La plus simple des cinq : une lecture de l'attribut booléen `only_allow_merge_if_pipeline_succeeds`. Elle sert de cas témoin, à la fois dans les tests unitaires et lors des validations en CI, car son comportement est prévisible et ne dépend d'aucun contexte. La lecture passe par `getattr` avec valeur par défaut plutôt que par un accès direct : l'attribut peut être absent de la représentation renvoyée pour certains types de projets.

3.3.5 Règle d'accès utilisateur : une refonte instructive

Cette règle a connu la refonte la plus profonde du projet, à la suite de l'identification d'une erreur de conception.

L'erreur initiale. La première implémentation court-circuitait la vérification en édition Premium : elle retournait systématiquement `compliant: True` avec la justification « GitLab gère nativement ». Cette formulation confondait deux mécanismes distincts :

- **l'appartenance à un projet** (rôle du membre) — disponible sur *toutes* les éditions, y compris Community ;
- **l'octroi d'un accès nommé sur une branche protégée**, indépendamment du rôle projet — réservé aux éditions Premium et Ultimate.

La conséquence était un faux positif silencieux : la règle ne vérifiait strictement rien et déclarait conforme un utilisateur qui, en réalité, n'avait aucun accès.

La clarification. La règle a été repositionnée pour ce qu'elle est réellement : un *contournement de l'édition Community*. Élever le rôle projet d'un utilisateur est le seul levier disponible en édition gratuite pour lui donner un accès en poussée ou fusion sur une branche protégée, faute de pouvoir cibler un utilisateur individuellement. En édition Premium, ce contournement devient inutile puisque l'accès peut être accordé directement au niveau de la branche.

La conception finale. Quatre décisions ont été prises. La **portée des branches devient un paramètre d'appel** plutôt qu'une valeur dérivée : l'outil n'impose aucune hypothèse sur les branches devant recevoir l'accès. Le **retour est toujours une liste**, jamais un dictionnaire seul — un élément par utilisateur en édition Community, un par couple (utilisateur, branche) en Premium — ce qui évite une branche conditionnelle dans l'orchestrateur. Le **support devient multi-utilisateurs**, avec regroupement de tous les utilisateurs d'une même branche dans une *unique* requête, ce qui réduit le nombre d'appels d'API. Enfin, le **type d'accès se choisit par branche** (push, merge ou both) et non globalement.

La signature finale traduit ces quatre décisions : `check_specific_user_access(project, user_ids, tier="free", branches=None)`, où `branches` est un dictionnaire du type `{"main": "both", "develop": "merge", "release/*": "push"}`, ignoré en édition Community.

Un détail d'implémentation mérite d'être signalé, car il est facile à manquer : la modification en édition Premium est **purement additive**. Aucune entrée existante n'est supprimée, contrairement à la correction des branches protégées qui, elle, calcule un différentiel complet. La raison est fonctionnelle : accorder un accès à un utilisateur ne doit pas retirer les accès par rôle déjà en place.

3.4 Couche de remédiation automatique

3.4.1 Principe et garde-fous

La détection sans correction laisse le travail manuel intact. Une couche de remédiation a donc été développée, centralisée dans un module unique `fixes.py` plutôt que dispersée dans chaque fichier de règle — décision explicite, afin de garder une vue d'ensemble de toute la remédiation en un seul endroit.

Deux garde-fous encadrent son activation :

- l'option `-fix` est nécessaire pour qu'une correction soit même envisagée ; sans elle, le comportement d'audit en lecture seule est strictement inchangé ;
- l'option `-dry-run`, qui requiert `-fix`, affiche les charges utiles qui *seraient* envoyées sans effectuer aucune écriture.

3.4.2 La découverte majeure : la limite d'édition

Le développement de la correction des branches protégées a produit le comportement le plus déroutant du stage.

L'approche initiale utilisait une modification en place : `PATCH /projects/:id/protected_branches/:name`. Le serveur répondait 200 OK. Le code concluait au succès. Et l'interface GitLab montrait que *rien n'avait changé*.

L'investigation dans la documentation officielle a livré l'explication :

- le point d'entrée `PATCH` n'accepte les champs `allowed_to_push` et `allowed_to_merge` — les seuls permettant de modifier les niveaux d'accès — que sur les éditions **Premium** et **Ultimate** ;
- sur l'édition **Community**, ces champs sont *silencieusement ignorés*, d'où le faux succès ;
- le point d'entrée `PATCH` ne supporte *aucun* champ simple équivalent : `push_access_level` et `merge_access_level` au singulier n'existent qu'en création (`POST`), pas en mise à jour ;
- pour les tags protégés, l'API n'a *jamaïs* exposé de `PATCH`, quelle que soit l'édition.

Conclusion technique

Sur GitLab Community Edition, il n'existe **aucun moyen supporté de modifier en place** les niveaux d'accès d'une branche déjà protégée. La seule voie est de *déprotéger puis reprotéger* (`DELETE` suivi de `POST`).

3.4.3 Deux stratégies sélectionnées par un paramètre

Plutôt que de se limiter à la stratégie compatible Community, les deux ont été implémentées et sont sélectionnées dynamiquement. Ce choix était justifié : la DSI dispose d'une licence Entreprise, alors que l'instance de développement était en édition Community.

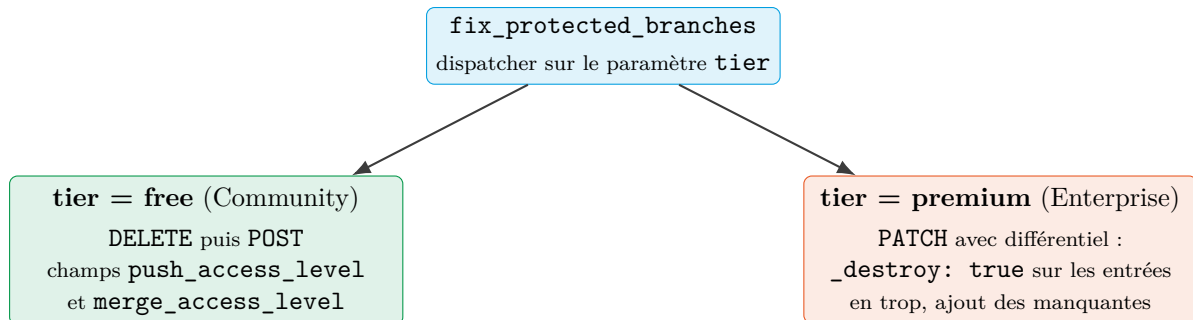


Figure 3.2 – Sélection de la stratégie de correction selon l'édition de GitLab

```

1 def _fix_protected_branches_free(project, result, dry_run):
2     branch_name = result.get("branch")
3     rule = next((r for r in EXPECTED_RULES
4                   if r["pattern"] == branch_name), None)
5     if rule is None:
6         return {..., "success": False,
7                   "details": f"Aucune règle définie pour '{branch_name}'"}
8
9     push_level = next(iter(rule["push"]))
10    merge_level = next(iter(rule["merge"]))
11
12    if dry_run:
13        return {..., "dry_run": True, "success": True,
14                  "details": "Dry-run : aucune écriture effectuée"}
15
16    # Aucun PATCH utilisable en CE : déprotéger puis reprotéger.
17    existing = next((pb for pb in project.protectedbranches.list(get_all=True)
18                     if pb.name == branch_name), None)
19    if existing is not None:
20        existing.delete()
  
```

```

21
22     project.protectedbranches.create({
23         "name": branch_name,
24         "push_access_level": push_level,
25         "merge_access_level": merge_level,
26     })

```

Extrait 3.4 – Correction des branches protégées en édition Community

Les deux autres corrections — tags protégés et contrainte de pipeline — sont identiques quelle que soit l'édition : la première parce qu'aucune modification en place n'existe pour les tags, la seconde parce qu'il s'agit d'un simple attribut booléen non concerné par la limitation.

3.4.4 Intégration dans le flux d'audit

Les résultats de correction sont accumulés dans la même liste plate que les résultats de vérification. Ils apparaissent donc naturellement dans les trois formats de rapport, sans traitement particulier. L'affichage console distingue les deux natures par des symboles différents.

3.5 Génération des rapports

Le module `report.py` expose trois générateurs orchestrés par une fonction unique, chacun consommant la même liste plate de dictionnaires.

Le générateur CSV pose une difficulté propre : les résultats n'ont pas tous les mêmes clés, puisque chaque règle peut en ajouter de spécifiques. La solution retenue construit dynamiquement l'en-tête à partir de l'union de toutes les clés rencontrées — les quatre clés communes en tête, les clés spécifiques triées ensuite — et s'appuie sur le paramètre `restval=""` de `csv.DictWriter` pour remplir les cellules absentes. Aucune modification n'est donc nécessaire lorsqu'une règle enrichit son résultat. Le rapport HTML, lui, applique une mise en forme par ligne selon le verdict et gère la différence de structure entre lignes de vérification et lignes de correction ; le contenu dynamique y est systématiquement échappé, un message d'erreur d'API pouvant contenir des caractères réservés.

Enfin, chaque exécution crée un dossier horodaté `reports/AAAA-MM-JJ_HH-MM-SS/` dans lequel les trois fichiers sont écrits. Ce détail, simple à implémenter, résout un vrai problème d'exploitation : sans lui, chaque exécution écrasait la précédente et rendait impossible toute comparaison dans le temps. Avec lui, la progression de la conformité du parc devient observable.

3.6 Industrialisation en intégration continue

3.6.1 Chaîne d'exécution

L'outil a été packagé (`pyproject.toml`, distribution `audit-gitlab-ci`) et intégré dans une chaîne GitLab CI. Le job est déclenchable manuellement via le formulaire ou automatiquement par planification, ce qui répond directement au symptôme de la dérive silencieuse identifié au chapitre 1.

La construction des arguments de la ligne de commande est faite en shell, de sorte que les options vides ne soient jamais transmises — un identifiant utilisateur non renseigné ne doit pas produire une option `--user-id` vide (l'extrait complet figure en annexe A). Deux points de vigilance y sont matérialisés. D'abord, la valeur par défaut de `AUDIT_DRY_RUN` est `true` : une activation de la correction sans précision supplémentaire simule, elle n'écrit pas. Ensuite, l'invocation utilise `python -m audit_tool.main` depuis la racine du dépôt, et non `python audit_tool/main.py` : cette dernière forme casse la résolution des imports du paquet.

Les rapports sont publiés en artefacts nommés d'après l'identifiant de la chaîne et conservés trente jours.

3.6.2 Bugs rencontrés lors de la mise en CI

Le passage du poste de développement à l'exécution en *runner* a révélé cinq problèmes, tous instructifs sur la différence entre les deux environnements. Le premier, `Neither 'setup.py' nor`

'`pyproject.toml`' found, venait de ce que le fichier de packaging n'existait que sur le poste Windows, jamais commité dans le clone Linux utilisé pour les poussées. Le correctif correspondant a ensuite été perdu, ayant été commité en état *detached HEAD* puis rendu orphelin par un `rebase -abort` : il a fallu le retrouver par `git reflog` et le réappliquer par `git cherry-pick`. Un `TypeError` sur une concaténation a révélé que la variable d'exclusion n'existait que dans le fichier `.env` local, ignoré par Git, et valait donc `None` en CI. Une espace manquante dans le fichier de chaîne collait l'identifiant utilisateur à l'option suivante, fusionnant deux arguments. Enfin, une erreur `Invalid header value` venait d'un retour à la ligne parasite dans la valeur du jeton, issu d'un copier-coller.

Enseignement

Aucun de ces cinq problèmes n'est un défaut de logique métier. Tous relèvent de l'écart entre l'environnement de développement et l'environnement d'exécution : fichiers non commités, variables locales absentes, données de configuration mal saisies. C'est précisément ce que l'intégration continue est censée révéler — et elle l'a fait.

3.6.3 Validation de bout en bout

Après correction, une exécution complète a été menée avec correction réelle — et non simplement simulée — sur quatre projets de test : installation propre du paquet, audit des quatre projets et vingt-et-un résultats collectés (vingt vérifications et une correction), rapports générés et publiés en artefacts. La seule non-conformité subsistante concernait les tags protégés d'un dépôt dépourvu de `pom.xml` comme de `package.json` : la correction a proprement refusé d'agir (« aucun motif de version détecté ») au lieu de deviner ou de planter, ce qui valide le comportement attendu sur les cas limites. Aucune régression n'a été introduite par la correction réelle sur les vérifications déjà conformes.

3.7 Tests et validation

3.7.1 Validation fonctionnelle

Table 3.2 – Scénarios de validation d'Audit-Tool

Scénario	Résultat
Audit en lecture seule, dépôt non conforme	Les trois familles de règles remontent les écarts attendus avec des messages exploitables
Simulation de correction (Community)	Charges utiles correctes pour les trois motifs de branches, aucune écriture constatée dans l'interface
Correction réelle (Community)	Les trois protections recréées avec les bons niveaux, vérifiées visuellement dans l'écran de configuration
Nouvel audit après correction	Les trois règles passent au vert : le cas positif est validé, en plus du cas négatif déjà couvert
Manifeste en sous-dossier	<code>pom.xml</code> détecté dans <code>my-app/</code> , motif correctement déterminé
Aucun manifeste	Non-conformité explicite ; la correction refuse d'agir plutôt que de deviner
Accès utilisateur — « ajout »	Utilisateur absent ajouté au projet avec le rôle Developer
Accès utilisateur — « ajustement »	Rôle rétrogradé manuellement à Guest, puis correctement réajusté à Developer
Utilisateur inexistant	Avertissement émis, règle ignorée, aucun appel d'API inutile
Édition Premium	Code écrit et cohérent avec la documentation, non validé faute d'accès à une instance Entreprise

3.7.2 Robustesse aux erreurs d'API

Un défaut de robustesse a été identifié puis corrigé : à l'origine, une exception non gérée — typiquement un `403 Forbidden` sur un projet où le compte du jeton n'avait pas le rôle suffisant — interrompait l'audit complet, empêchant le traitement des projets suivants.

La correction encadre *chaque appel de vérification individuellement*, et non la boucle entière, par un gestionnaire d'exceptions ciblé. Une fonction d'assistance convertit alors l'erreur en un résultat conforme à la convention, avec le code HTTP et le message d'erreur dans le champ de détails — l'incident apparaît ainsi dans le rapport au lieu d'interrompre l'exécution.

Le choix de capturer `gitlab.exceptions.GitlabError` plutôt que `Exception` est délibéré : un `except Exception` masquerait les vrais bugs de programmation au même titre que les erreurs de plateforme, ce qui est exactement le comportement à éviter.

Conclusion du chapitre

Audit-Tool répond aux objectifs O1 à O4 : il parcourt une instance complète, évalue cinq règles, produit trois formats de rapport historisés, corrige optionnellement les écarts avec deux stratégies selon l'édition, et s'exécute en intégration continue.

Il conserve toutefois une limite structurelle : il constate et corrige *a posteriori*. Un projet créé aujourd'hui sera non conforme jusqu'à la prochaine exécution planifiée. C'est cette limite qui a motivé le second projet.

CHAPITRE 4

GovernX : provisionnement de projets conformes par construction

Introduction du chapitre

Le chapitre précédent s’achevait sur une limite : Audit-Tool corrige *a posteriori*. GOVERNX répond à cette limite en déplaçant le contrôle en amont, au moment de la création du projet. Ce chapitre décrit sa genèse, son architecture en trois axes, le détail de sa chaîne de provisionnement en trois étapes, les décisions d’architecture qui l’ont façonné, et sa stratégie de test.

4.1 Genèse et positionnement

4.1.1 Du constat à la prévention

Lorsqu’une équipe applicative crée un nouveau dépôt chez CIH Bank, elle doit reproduire une configuration attendue : structure de branches conforme au modèle interne, protections adéquates, *topics* de classification, squelette applicatif correspondant à la technologie retenue. Rien n’empêche l’oubli, et c’est pourtant sur ce point de départ que reposent toutes les garanties ultérieures.

L’idée de GOVERNX tient en une phrase : *industrialiser la création pour que la conformité ne soit plus une action, mais une propriété.*

Principe fondateur

GOVERNX ne réimplémente pas les règles de conformité : il *consomme* celles d’Audit-Tool. Deux implémentations d’une même règle divergent inmanquablement au premier correctif appliqué d’un seul côté. Ce principe de non-duplication a été appliqué avec rigueur — et rappelé explicitement dans la documentation d’architecture chaque fois qu’une nouvelle couche était envisagée.

4.1.2 Une architecture en trois axes

GOVERNX a été conçu comme un ensemble de trois axes complémentaires. Seul le premier a été réalisé et validé durant le stage ; les deux autres ont été cadrés et planifiés, et sont présentés en fin de chapitre comme perspectives assumées.

L’**axe 1** constitue le socle métier : la chaîne CI/CD de création, de tagging, d’audit et de correction. L’**axe 2** est une *couche d’interface* au-dessus de lui, sous la forme d’un portail développeur en libre-service. L’**axe 3** est une *couche de packaging et de sécurité* autour de lui : conteneurisation de l’outil d’audit et analyse de vulnérabilités. Ni l’un ni l’autre ne duplique la logique métier — ils la consomment — ce qui explique que la stabilisation de l’axe 1 conditionnait leur démarrage.

4.2 Axe 1 : la chaîne de provisionnement

4.2.1 Vue d’ensemble

L’utilisateur déclenche la chaîne depuis le formulaire natif de GitLab en renseignant quatre informations : le ou les noms de projets à créer, le type de projet, le groupe cible, et le mode de correction automatique souhaité. La chaîne enchaîne alors trois étapes.

4.2.2 Le garde-fou de déclenchement

Avant toute logique métier, un garde-fou global limite la création même de la chaîne au déclenchement manuel par formulaire. Cette précaution est proportionnée à l’effet de bord : la chaîne *crée des*

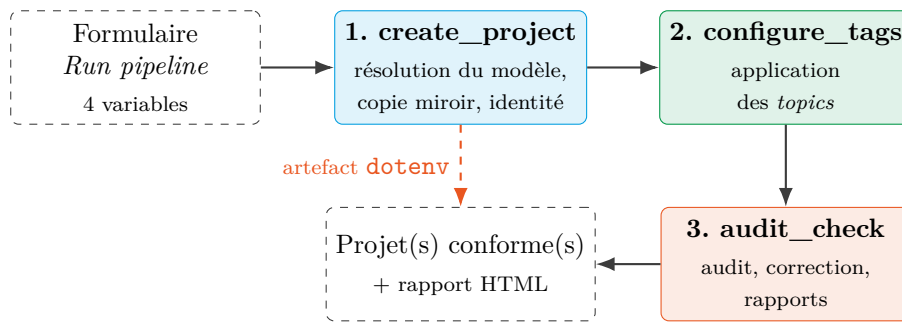


Figure 4.1 – Les trois étapes de la chaîne GVERNAX

projets. Un déclenchement involontaire par simple poussée de code produirait des dépôts parasites dans le groupe cible.

4.2.3 Étape 1 : résolution du modèle et création

Résolution par *topic*. Plutôt que de maintenir dans le code un tableau associant chaque type de projet à un dépôt modèle — tableau qu’il faudrait modifier et redéployer à chaque ajout au catalogue — GVERNAX interroge GitLab : il recherche, dans le groupe des modèles, les projets portant le *topic* correspondant au type demandé. Le catalogue de modèles devient ainsi gérable depuis l’interface GitLab, sans toucher au code.

La sélection applique une politique stricte :

- **zéro correspondance** → échec immédiat avec un message explicite. Créer un projet vide « au cas où » serait pire que ne rien créer ;
- **une correspondance** → sélection directe ;
- **plusieurs correspondances** → échec, sauf si une variable de désambiguïsation est fournie explicitement. Cette variable est comparée par **égalité stricte** au nom ou au chemin du candidat, jamais par sous-chaîne : un futur modèle au nom proche casserait silencieusement une désambiguïsation qui fonctionnait auparavant.

```

1 def select_template(candidates, project_type, variant):
2     if not candidates:
3         raise TemplateResolutionError(
4             f"aucun template trouve pour PROJECT_TYPE='{project_type}'")
5     if len(candidates) == 1:
6         return candidates[0]
7     if not variant:
8         raise TemplateResolutionError(
9             f"plusieurs templates correspondent a '{project_type}' "
10            "Fournis TEMPLATE_VARIANT pour desambiguiser.")
11
12     # Egalite STRICTE, jamais de sous-chaîne.
13     variant_lower = variant.lower()
14     filtered = [c for c in candidates
15                if c.get("name", "").lower() == variant_lower
16                   or c.get("path", "").lower() == variant_lower]
17     if len(filtered) != 1:
18         raise TemplateResolutionError(
19             f"TEMPLATE_VARIANT='{variant}' ne correspond pas exactement "
20             "a un unique candidat.")
21     return filtered[0]

```

Extrait 4.1 – Sélection du modèle avec politique stricte

Architecture *cross-instance*. Une contrainte d’environnement a structuré cette étape : les dépôts modèles sont hébergés sur GitLab.com, tandis que les nouveaux projets sont créés sur l’instance auto-hébergée qui exécute la chaîne. Deux clients d’API distincts coexistent donc, chacun avec son serveur et son jeton : un client *source* en lecture seule sur GitLab.com, et un client *destination* en écriture sur l’instance locale. Le *runner* joue le rôle de pont — il clone le modèle depuis la première

instance et pousse vers la seconde. Cette séparation a un bénéfice de sécurité direct, développé au chapitre 5 : un jeton compromis n'a qu'un pouvoir borné.

Méthode de copie. Trois méthodes étaient envisageables pour recopier un modèle : la bifurcation (*fork*) suivie d'un détachement, l'export/import natif de GitLab, ou la copie miroir Git. La troisième a été retenue :

- elle ne laisse **aucune métadonnée résiduelle** liant le nouveau projet au modèle ;
- elle fonctionne **identiquement** sur les éditions Community et Enterprise ;
- elle copie **toutes** les branches et tous les tags, contrairement à une poussée simple qui ne prendrait que la branche par défaut.

Un test dédié a été mené pour caractériser précisément ce que cette méthode transporte, en créant un dépôt d'essai à cinq branches. Le résultat est net et important pour la suite.

Table 4.1 – Ce que la copie miroir transporte réellement

	Contenu Git (<i>copié</i>)	Réglage GitLab (<i>jamais copié</i>)
Branches	Toutes les branches et leur historique	Les protections de branches, stockées en base de données
Tags	Tous les tags Git existants	Les protections de tags, stockées en base de données

Conséquence architecturale

Un projet fraîchement créé possède bien toutes les branches et tous les tags du modèle, mais **aucune protection**. C'est donc l'étape 3 qui doit les configurer via l'API, indépendamment de ce qui existait sur le modèle source. Ce comportement est voulu : les règles de protection viennent du référentiel de l'établissement, et non d'une copie de la configuration d'un modèle qui pourrait elle-même avoir dérivé.

Traitement par lot. Le champ des noms de projets accepte plusieurs valeurs, séparées par des virgules, des points-virgules ou des retours à la ligne. Trois optimisations en découlent. Le modèle n'est cloné **qu'une seule fois**, puis poussé vers chaque destination — une poussée avec URL explicite ne modifiant pas le dépôt local, plusieurs poussées concurrentes depuis le même miroir sont sûres. Les projets sont provisionnés **en parallèle**, avec un degré de parallélisme configurable. Enfin, l'échec d'un projet **n'interrompt pas** les autres : la chaîne doit dire ce qui a été créé et ce qui ne l'a pas été, plutôt que de s'arrêter au premier problème — ce qui a justifié le remplacement des sorties abruptes du client REST par une exception dédiée, rattrapable projet par projet. Une validation préalable refuse par ailleurs deux noms produisant le même identifiant d'URL (« Mon Projet » et « mon-projet ») *avant* toute écriture.

Alignement de l'identité du projet. Juste après la copie, la chaîne réécrit le champ identifiant du manifeste de build sur un identifiant dérivé du nom du projet : la balise `<artifactId>` racine du `pom.xml` pour un projet Maven, la clé `"name"` racine du `package.json` pour un projet npm. « Simple SpringBoot App » devient ainsi `simple-springboot-app`.

Cette réécriture est délicate : un `pom.xml` contient typiquement plusieurs balises `<artifactId>` — celle du projet, mais aussi celle du parent Spring Boot et celle de chaque dépendance. Un remplacement naïf par expression régulière corromprait le fichier. L'implémentation procède donc en deux temps :

1. une **édition textuelle ciblée** avec suivi de profondeur, qui ne retient que la balise située au niveau racine du projet. Les commentaires XML sont préalablement neutralisés en préservant les décalages, de sorte qu'un `artifactId` en commentaire ne puisse jamais être sélectionné ;

2. une **relecture structurelle de contrôle**, qui vérifie que la réécriture a bien produit la valeur voulue et n'a altéré aucun autre champ.

```

1 def _find_root_artifact_id(text):
2     """Localise le <artifactId> enfant direct de <project>.
3
4     Le suivi de profondeur garantit qu'on ne remonte jamais l'artifactId
5     d'un <parent> ou d'une <dependency>, qui sont à une profondeur
6     strictement supérieure.
7     """
8     scan = _strip_comments(text) # commentaires neutralisés, offsets preserves
9     depth = 0
10
11     for m in _TAG_RE.finditer(scan):
12         closing, tag, _attrs, self_closing = m.groups()
13         local = tag.split(":")[-1]
14         if closing:
15             depth -= 1
16             continue
17         if not self_closing and local == "artifactId" and depth == 1:
18             end = scan.find("<", m.end())
19             if end == -1:
20                 break
21             return m.end(), end, text[m.end():end].strip()
22         if not self_closing:
23             depth += 1
24
25     raise ManifestError(
26         "pom.xml sans <artifactId> au niveau racine du projet "
27         "(un artifactId de <parent> ou de <dependency> ne compte pas).")

```

Extrait 4.2 – Localisation de la balise racine par suivi de profondeur

Le même principe est appliqué au `package.json` : un automate suit la profondeur des accolades, de sorte qu'une clé "name" imbriquée — celle de l'auteur, par exemple — soit à une profondeur supérieure à un et ne puisse donc jamais être sélectionnée.

Le commit est écrit via l'API GitLab, sans aucun clone local : le clone miroir reste cantonné à la copie initiale du modèle.

4.2.4 Étape 2 : application des *topics*

Cette étape est volontairement simple. Elle applique aux projets créés deux *topics* déterminés par le type de projet — le type lui-même et la technologie associée, lue dans un tableau de correspondance figé — et rien d'autre.

Cette simplicité résulte d'une décision : à l'origine, cette étape créait aussi le tag protégé et embarquait une logique d'arbitrage entre technologies. Un cadrage avec l'encadrant a confirmé qu'*aucune ambiguïté n'est possible* — chaque type de projet correspond à exactement une technologie et un manifeste. Toute la logique d'arbitrage a donc été retirée, et la création du tag protégé déplacée vers l'étape d'audit, là où réside la logique de conformité.

Table 4.2 – Correspondance type de projet, technologie et manifeste

Type	Technologie	Manifeste	Champ identifiant
backend	Spring Boot	pom.xml	<artifactId> racine
batch	Spring Boot	pom.xml	<artifactId> racine
dependencie	Spring Boot	pom.xml	<artifactId> racine
frontend	React	package.json	clé "name" racine

4.2.5 Étape 3 : audit et correction

C'est ici que GOVERNX rejoint Audit-Tool. L'étape instancie une connexion, récupère chaque projet créé, exécute les trois règles applicables, et applique les corrections si le mode automatique est demandé.

```

1 def run_checks(project):

```

```

2  results = []
3  results.extend(check_protected_branches(project))
4  results.append(check_protected_tags(project))
5  results.append(check_pipeline_success(project))
6  return results
7
8
9  def audit_project(project, auto_fix, tier):
10     results = run_checks(project)
11     fixes = []
12
13     if auto_fix:
14         fixes = apply_fixes(project, results, tier=tier)
15         results = run_checks(project) # re-verification apres correction
16
17     return {
18         "project": project.path_with_namespace,
19         "compliant": all(r["compliant"] for r in results),
20         "results": results,
21         "fixes": fixes,
22     }

```

Extrait 4.3 – Orchestration de l'audit d'un projet

Le point à souligner est la **re-vérification systématique après correction**. Le rapport ne reflète jamais l'état antérieur à la remédiation, et surtout, une correction qui se déclare réussie sans que le nouvel état le confirme est détectée. Ce détail apparemment mineur a été décisif : c'est lui qui a révélé le bug de modélisation des niveaux d'accès analysé au chapitre 5.

Périmètre réduit et règle écartée. Trois règles sur cinq sont reprises. La règle d'accès utilisateur est explicitement hors périmètre : elle concerne un utilisateur donné appliqué à *tous* les projets d'une instance, et non la conformité intrinsèque du projet créé.

Comportement *fail-closed*. Si un projet est non conforme et qu'aucune correction n'a été demandée, la chaîne **bloque** avec un message explicite. Elle ne peut continuer que si l'opérateur fournit explicitement une variable d'acceptation — ce qui laisse une trace de la décision dans les paramètres d'exécution. Ce comportement a été validé sur plusieurs projets de test : cinq règles sur cinq détectées non conformes sur un projet fraîchement créé, aucune correction tentée, chaîne bloquée proprement, rapport tout de même généré.

Isolation des erreurs. Un projet injoignable produit un bloc de rapport non conforme plutôt que de faire échouer l'étape entière : le rapport doit couvrir l'ensemble du lot. Une précaution de concurrence a également été prise — la bibliothèque cliente n'offrant aucune garantie de sûreté vis-à-vis des fils d'exécution, une connexion distincte est instanciée par fil.

4.2.6 Le rapport d'audit

L'étape produit trois artefacts : un rapport JSON exploitable par un outil tiers, un rapport texte pour la lecture en console, et un rapport HTML autonome exposé directement sur la page de la chaîne dans l'interface GitLab. Ce dernier format répond à un besoin concret : permettre à un responsable non technique de consulter le résultat sans télécharger d'artefact ni lire du JSON.

Le rapport couvre l'ensemble du lot : une vue d'ensemble, puis une section par projet détaillant les vérifications et les corrections appliquées.

4.3 Décisions d'architecture

Le développement de GOVERNIX a donné lieu à quinze décisions d'architecture formellement consignées, datées et justifiées. Les tableaux qui suivent en présentent les plus structurantes, y compris celles qui ont été ultérieurement remplacées — leur trace est instructive, car elle documente pourquoi une approche a été abandonnée.

Table 4.3 – Décisions d'architecture structurantes

N°	Décision	Justification
2	Échec dur sur zéro ou multi-correspondance de modèle	Créer un projet vide ou choisir arbitrairement entre deux modèles produirait un résultat inattendu et silencieux
3	Rapport toujours généré, blocage sauf acceptation explicite	La traçabilité de l'acceptation d'un projet non conforme est une exigence de gouvernance
5	Tout en Python, aucun script Bash	Cohérence avec la logique métier, testabilité homogène
6	Garde-fou global plutôt que validation par job	Empêche la <i>création même</i> d'une chaîne hors formulaire, et non seulement son exécution
7	Aucun dossier de modèles local	Les modèles vivent sur GitLab ; une copie de référence dans le dépôt divergerait
8	Suppression de tout mécanisme de repli	Un repli masque l'absence de logique métier réelle et donne l'illusion du fonctionnement
9	Désambiguïsation par égalité stricte	Une correspondance par sous-chaîne casserait dès l'ajout d'un modèle au nom proche
10	Correspondance déterministe type → manifeste	Confirmé par l'encadrant : aucune ambiguïté possible. Supprime toute la logique d'arbitrage précédente
12	Réutilisation des règles réelles d'Audit-Tool	Évite deux implémentations divergentes de la même règle
13	Architecture <i>cross-instance</i>	Contrainte d'environnement : les modèles vivent sur GitLab.com, la création se fait sur l'instance locale
14	Niveau d'accès modélisé comme un seuil unique	L'API hérite automatiquement des rôles supérieurs ; une liste de rôles ne persiste pas
15	Motif de tag dérivé du <i>contenu</i> du manifeste	Un motif générique protégerait indistinctement tous les tags ; un motif dérivé de l'identité du projet est spécifique

4.3.1 Focus sur la décision 15 : motif dérivé du contenu

Cette dernière décision mérite un développement, car elle illustre une manière de raisonner autant qu'un choix technique.

Le motif de tag protégé était initialement générique, déduit du *type* de manifeste : `*` pour un projet Maven, `v*` pour un projet npm. Ce motif a le mérite de la simplicité mais un défaut : il ne distingue rien. Tous les projets Maven partagent le même motif.

La décision 15 le remplace par un motif dérivé du *contenu* du manifeste : `{artifactId}-*`, soit `simple-springboot-app-*` pour notre exemple. Le motif devient propre au projet.

Le point de conception le plus intéressant concerne la **source de vérité**. L'identifiant est calculé à l'étape 1, mais l'étape 3 ne le réutilise *pas* : elle relit le manifeste. Ce choix est délibéré, et la justification vaut d'être citée : les deux étapes sont des jobs CI distincts, et *repasser par le fichier garde les deux étapes synchronisées par construction plutôt que par convention*. Si un développeur modifie le manifeste entre-temps, c'est le fichier qui fait foi.

Conséquence assumée

Un projet Maven *multi-modules* — un POM parent et un POM par module — est refusé par cette conception, car rien ne permet de désigner sans deviner le module porteur de l'identité du projet. Cette limite est documentée explicitement plutôt que contournée par une heuristique. Elle sera à revoir si un modèle multi-modules entre au catalogue.

4.3.2 Le principe *fail-closed* en pratique

Ce principe traverse tout GOVERNIX. Six situations d'ambiguïté conduisent à un échec explicite plutôt qu'à une valeur devinée :

1. aucun manifeste trouvé dans l'arborescence ;
2. plusieurs manifestes candidats (cas multi-modules) ;
3. deux technologies concurrentes détectées dans le même dépôt ;
4. champ identifiant absent, vide ou illisible ;
5. manifeste syntaxiquement invalide ;
6. branche par défaut du projet indéterminée.

Dans chacun de ces cas, la vérification retourne un verdict non conforme accompagné de la cause précise, et la correction n'écrit rien.

4.4 Stratégie de test

4.4.1 Trois niveaux

Les tests s'organisent en trois niveaux, du plus autonome au plus représentatif. Les **tests unitaires** — 84 au total, sans aucune dépendance réseau — couvrent les identifiants, la réécriture de manifeste, le motif dérivé du contenu, les cas d'échec contrôlé et le renommage par projet dans un lot. L'**intégration partielle** valide l'étape de création seule contre un vrai groupe GitLab d'essai. Enfin, la **chaîne complète en CI** enchaîne les trois étapes en conditions réelles, sur une instance dotée d'un *runner*.

4.4.2 Une leçon sur les tests

Un enseignement mérite d'être rapporté honnêtement. La suite de tests unitaires a été perdue puis recrée : elle avait été écrite et validée localement, mais jamais commitée. Pendant plusieurs semaines, la chaîne a donc tourné sans filet de sécurité, et cette absence n'a été détectée qu'à l'occasion d'une réconciliation entre la documentation et l'état réel du dépôt.

Plus instructif encore : après recréation, l'analyse a conclu que les deux bugs les plus graves du projet — la modélisation des niveaux d'accès et le nom de branche principale codé en dur — *n'auraient probablement pas été détectés* par les tests initiaux, dont les objets simulés étaient trop permissifs. Un objet simulé qui accepte tout ne teste rien. Les tests ont donc été renforcés avec des simulacres plus stricts et un cas de branche par défaut différente de `main`.

Les jeux de données de test ont été construits comme autant de pièges volontaires plutôt que comme des cas idéaux : un `pom.xml` contenant un `<artifactId>` sous `<parent>` et sous `<dependency>`, de part et d'autre du véritable identifiant du projet ; un `package.json` comportant une clé `"name"` imbriquée sous `author` ; un manifeste syntaxiquement invalide. Un test qui ne passe que sur le cas nominal ne prouve rien de la robustesse du code.

4.4.3 Optimisation de la chaîne

La chaîne a fait l'objet d'un travail d'optimisation, motivé par un temps d'exécution jugé excessif :

- passage à une image Docker embarquant déjà Git, ce qui supprime une installation de paquet d'environ trente secondes à chaque job ;
- mise en cache des dépendances Python, avec une clé dérivée du fichier de dépendances ;
- clone superficiel du dépôt de la chaîne, dont l'historique est inutile ;
- archivage des artefacts en parallèle ;
- déduplication du fichier de configuration, dont le contenu était présent deux fois.

4.5 Perspectives : les axes 2 et 3

Axe 2 — Portail développeur Backstage. L'objectif est de remplacer le formulaire GitLab par un portail en libre-service. Un plan en cinq phases a été établi, la phase zéro consistant en une démonstration locale minimale : une action de *scaffolder* déclenchant la chaîne GOVERN

par appel d'API, et un modèle de formulaire. Un avertissement explicite a été inscrit dans la documentation d'architecture, révélateur de la discipline appliquée au projet : *si les actions de scaffoldier réimplémentent la logique métier en TypeScript au lieu d'appeler l'API pour déclencher la chaîne existante, c'est le même problème de duplication que celui corrigé côté règles d'audit*. Backstage doit **déclencher** l'axe 1, pas le dupliquer.

Axe 3 — Conteneurisation et analyse de vulnérabilités. L'objectif est d'empaqueter l'outil d'audit dans une image Docker et d'en analyser les vulnérabilités. Les livrables produits sont un `Dockerfile` multi-étapes avec utilisateur non privilégié, un fichier d'exclusion de contexte de construction, et un fragment de chaîne comportant les jobs de construction, d'analyse et de publication. Deux points restent ouverts et sont assumés comme tels : la politique d'exclusion des vulnérabilités non pertinentes — une image de base contient inévitablement des vulnérabilités système sans rapport avec l'application — et la gestion des secrets, qui devrait passer par un coffre-fort dédié plutôt que par des variables de CI.

Conclusion du chapitre

GOVERNIX répond aux objectifs O5 et O6 : il provisionne un ou plusieurs projets depuis un modèle validé, les configure, les audite et les corrige en une seule exécution, sans dupliquer la logique métier d'Audit-Tool. Son comportement est *fail-closed* sur six situations d'ambiguïté distinctes.

Les deux projets forment ainsi un ensemble cohérent : GOVERNIX garantit qu'un projet naît conforme, Audit-Tool vérifie périodiquement qu'il le reste. Le chapitre suivant dresse le bilan de ce travail et analyse en détail les bugs les plus instructifs rencontrés.

CHAPITRE 5

Bilan, difficultés et perspectives

Introduction du chapitre

Ce dernier chapitre dresse le bilan du travail réalisé. Il commence par les résultats mesurables, puis consacre une part importante à l'analyse détaillée des bugs les plus instructifs rencontrés — un choix délibéré, car ce sont eux qui ont le plus appris. Il aborde ensuite les considérations de sécurité, les limites connues assumées, et enfin les perspectives d'évolution.

5.1 Résultats obtenus

5.1.1 Bilan quantitatif

Les deux livrables totalisent une vingtaine de modules Python — huit pour Audit-Tool, douze pour GOVERNX — pour environ 3 500 lignes de code. Cinq règles de conformité distinctes ont été implémentées, dont trois sont réutilisées telles quelles par GOVERNX, accompagnées de trois fonctions de remédiation déclinées en deux stratégies selon l'édition de GitLab. S'y ajoutent trois formats de rapport, 84 tests unitaires sans dépendance réseau, quatre étapes de chaîne CI/CD réparties sur les deux projets, et quinze décisions d'architecture formellement consignées.

5.1.2 Bilan par objectif

Table 5.1 – Atteinte des objectifs fixés au chapitre 1

Réf.	Objectif	Statut	Commentaire
O1	Auditer une instance complète	Atteint	Validé sur l'instance de développement, quatre projets audités en une exécution
O2	Rapport exploitable	Atteint	CSV, JSON et HTML, historisés par exécution horodatée
O3	Correction automatique	Atteint	Validé en édition Community ; code Premium écrit mais non testé faute d'accès à une instance Entreprise
O4	Industrialisation	Atteint	Chaîne validée de bout en bout avec correction réelle
O5	Provisionnement conforme	Partiellement	Chaîne complète fonctionnelle, comportement <i>fail-closed</i> validé ; une exécution intégralement verte avec correction automatique restait à confirmer en fin de stage
O6	Non-duplication de la logique métier	Atteint	Les règles ne sont écrites qu'une fois et réutilisées par GOVERNX

L'honnêteté impose de détailler le statut partiel de l'objectif O5. La chaîne fonctionne : chacune de ses trois étapes a été validée individuellement en conditions réelles, et le comportement de blocage sur non-conformité a été confirmé sur plusieurs projets. Ce qui restait à confirmer en fin de stage était une exécution intégralement verte avec correction automatique activée, après application de l'ensemble des correctifs de la dernière itération. Cette nuance est importante et n'est pas masquée : elle constitue le premier élément de la liste des travaux à poursuivre.

5.2 Analyse des bugs marquants

Cinq bugs méritent une analyse détaillée. Ils ont en commun de ne pas être des erreurs de syntaxe ou d'inattention, mais des *erreurs de modèle mental* : le code faisait exactement ce qu'on lui demandait, la demande était fausse.

5.2.1 Bug 1 — Le seuil pris pour une liste

Symptôme. La correction des branches protégées se déclarait réussie, mais la re-vérification échouait systématiquement sur `develop` et `release/*`. La branche principale, elle, passait.

Investigation. L'asymétrie était la clé. Les deux branches en échec avaient en commun d'attendre *deux* rôles distincts pour la fusion (Developer et Maintainer), là où la branche principale n'en attendait qu'un.

Cause. Le modèle attendu était exprimé comme un *ensemble de rôles explicites*, $\{30, 40\}$, alors que l'API GitLab modélise l'accès comme un *seuil unique* : un rôle donné inclut automatiquement tous les rôles supérieurs. La correction envoyait donc deux entrées, dont une seule était persistée par GitLab. La comparaison $\{40\} \neq \{30, 40\}$ échouait indéfiniment.

Correction. Passage à une valeur unique par branche et par action, et usage des champs d'API de seuil simple plutôt que des champs de liste — ces derniers étant réservés aux règles granulaires par utilisateur des éditions commerciales.

Ce que ce bug enseigne

La documentation de l'API décrit une *forme* de données, pas une *sémantique*. Les deux modélisations produisent un JSON valide et acceptable par le serveur ; seule l'une des deux correspond à ce que GitLab fait réellement. La vérification après correction est ce qui a rendu l'écart visible : sans elle, le code aurait continué à rapporter des succès.

5.2.2 Bug 2 — Le correctif partiel

Symptôme. Après correction du bug 1, une exception à l'exécution : `TypeError: 'int' object is not iterable`.

Cause. Le passage d'un ensemble à un entier avait rendu invalide un appel de tri appliqué à ces valeurs dans le message d'action. Le premier passage de correction n'avait retiré cet appel qu'à *un* des deux endroits — celui qui apparaissait dans la trace de la première erreur. Le second est resté, et a produit exactement la même exception au run suivant.

Ce que ce bug enseigne

Corriger l'occurrence signalée par la trace n'est pas corriger le bug. Lorsqu'un changement de type invalide un motif de code, il faut rechercher *toutes* les occurrences de ce motif, pas seulement celle qui a planté en premier.

5.2.3 Bug 3 — Le faux positif le plus dangereux

Ce bug est le plus intéressant des cinq, car il produisait un résultat *apparemment correct*.

Symptôme. Sur un projet réel, la vérification des tags protégés échouait avec « aucun manifeste trouvé », alors que le projet, créé depuis un modèle backend, en possédait un.

Investigation. La vérification cherchait le manifeste sur la branche `main`. Or le projet avait `master` comme branche par défaut, et `main` *n'existait pas du tout* dans le dépôt. La lecture échouait donc silencieusement, ce qui donnait l'illusion d'une absence de manifeste.

Le vrai problème. L'investigation a révélé un second effet, bien plus grave, sur une autre règle. La correction des branches protégées ciblait elle aussi `main` en dur. Elle avait « réussi » à créer une règle de protection sur cette branche — car **GitLab accepte de créer une règle de protection sur un nom de branche qui n'existe pas**. La re-vérification trouvait donc la règle présente et déclarait la branche conforme.

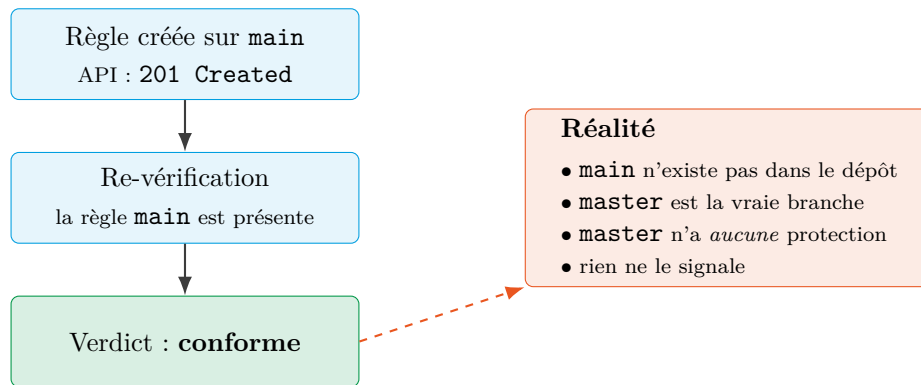


Figure 5.1 – Mécanisme du faux positif : une protection créée sur une branche inexistante

Conséquence. Le système rapportait « conforme » pendant que la véritable branche principale restait totalement dépourvue de protection. C’est le pire résultat possible pour un outil de conformité : non pas une erreur visible, mais une *fausse assurance*.

Correction. Remplacement du nom en dur par une détection dynamique de la branche par défaut réelle du projet. Si celle-ci est indéterminée, la vérification échoue explicitement plutôt que de deviner un nom.

Ce que ce bug enseigne

Un outil de conformité qui produit un faux positif est plus dangereux qu’un outil absent : il crée une confiance injustifiée. Toute hypothèse codée en dur sur l’environnement — un nom de branche, un chemin, une convention — est un faux positif en attente. La détection dynamique, avec échec explicite si la détection est impossible, est la seule position défendable.

5.2.4 Bugs 4 et 5 — Deux illusions symétriques

Le succès qui ne change rien. Décrit au chapitre 3 : la modification en place d’une branche protégée renvoyait 200 OK sans rien modifier sur l’édition Community. La recherche de la cause dans le code était vaine, puisque le code était correct au regard de la documentation générale ; seule la lecture attentive des mentions d’édition a livré l’explication. Un code de retour HTTP de succès atteste qu’une requête a été *acceptée*, pas qu’un effet a été *produit* : sur une API dont le comportement varie selon l’édition, la seule vérification fiable est de relire l’état après écriture.

Le code qui ne s’applique pas. Après correction d’une fonction, les anciens journaux continuaient d’apparaître à l’exécution, malgré une modification visible dans l’éditeur. Le dossier du projet était synchronisé par un service de stockage en ligne, et le fichier marqué « disponible à la demande » : la sauvegarde ne prenait pas effet dans le fichier réellement chargé par l’interpréteur. Face à un comportement du type « le code ne semble pas s’appliquer », le réflexe doit être de vérifier *ce que l’interpréteur charge réellement* — chemin du module et code source en mémoire — avant de suspecter la logique. Chercher un bug dans du code qui n’est pas celui qui s’exécute est une perte de temps garantie.

5.2.5 Synthèse

5.3 Considérations de sécurité

5.3.1 Gestion des jetons

Un jeton d’API GitLab de scope complet est un secret sensible : il permet de lire tout le code accessible au compte porteur et, pour les besoins de la remédiation, d’en modifier la configuration. Plusieurs mesures ont été appliquées :

Table 5.2 – Synthèse des cinq bugs marquants

Bug	Nature	Principe retenu
Seuil / liste	Modèle mental erroné de l'API	Vérifier après écriture, ne jamais se fier au code de retour seul
Correctif partiel	Correction incomplète d'un motif	Chercher toutes les occurrences, pas seulement celle de la trace
Branche en dur	Hypothèse implicite sur l'environnement	Détecter dynamiquement, échouer explicitement si impossible
Succès sans effet	Comportement dépendant de l'édition	Lire la documentation en s'attachant aux mentions d'édition
Code non appliqué	Environnement de développement	Vérifier ce que l'interpréteur charge réellement

- toutes les variables contenant un jeton sont déclarées **masquées et protégées**, au niveau du groupe racine plutôt que projet par projet ;
- les jetons sont **expurgés** des sorties des commandes Git avant affichage dans les journaux, une URL de poussée authentifiée contenant le jeton en clair ;
- l'architecture *cross-instance* sépare deux jetons de portées différentes : lecture seule sur l'instance des modèles, écriture sur l'instance de destination. Un jeton compromis a ainsi un pouvoir borné.

5.3.2 Incidents et vigilance

Deux points de vigilance sont rapportés en toute transparence, car ils constituent des enseignements réels :

1. **Des jetons ont été collés en clair** lors de sessions de débogage. Bien qu'il s'agisse de jetons d'un environnement de développement isolé, la pratique est à proscrire, et la révocation puis la régénération de ces jetons a été inscrite dans la liste des actions à mener.
2. **Le passage du jeton en argument de ligne de commande** le rend visible dans l'historique du shell et dans la liste des processus, contrairement à un fichier d'environnement. Ce choix, fait délibérément pour rendre les démonstrations plus explicites, est acceptable en environnement de test mais devra être revu si l'outil s'exécute un jour sur un serveur partagé.

5.3.3 Principe du moindre privilège

Une observation importante pour une future mise en production a été formalisée : le scope du jeton ne suffit pas. Le compte de service porteur devra disposer du rôle Maintenir ou Owner sur les projets concernés — l'énumération des branches protégées l'exige — et non simplement d'un jeton correctement scopé. Ce point est resté ouvert et doit être arbitré avec l'équipe infrastructure.

5.3.4 Sensibilité des données de gouvernance

Une réflexion a été menée sur une éventuelle version *cloud-native* des outils. Elle a conduit à une position prudente : les données de gouvernance d'un établissement bancaire — inventaire des dépôts, règles de conformité, écarts constatés — ne doivent pas transiter vers un cloud public sans validation préalable des équipes sécurité et conformité, au regard notamment de la réglementation marocaine sur la protection des données personnelles.

La proposition formulée sépare donc explicitement deux environnements : l'audit réel reste entièrement interne, et une éventuelle démonstration *cloud-native* tournerait sur un compte personnel avec une instance GitLab de test et des données factices, uniquement pour prouver que l'architecture fonctionne.

5.4 Limites connues

L'honnêteté d'un bilan se mesure à ce qu'il reconnaît ne pas avoir traité.

Table 5.3 – Limites connues et assumées

Limite	Analyse
Édition Premium non validée	Le code de correction pour les éditions commerciales est écrit et cohérent avec la documentation, mais n'a jamais été exécuté sur une instance Entreprise
Projets Maven multi-modules refusés	Conséquence assumée du choix de dériver l'identité du contenu du manifeste : rien ne permet de désigner sans deviner le module porteur de l'identité
Niveau d'accès des tags protégés à confirmer	Un écart existe entre la valeur envoyée et ce que la documentation de l'API décrit pour ce point d'entrée. Il faut relire l'état après une correction réussie pour trancher
Deux conventions d'identifiant coexistent	L'identifiant du chemin GitLab et celui du manifeste traitent différemment la ponctuation. Non unifié volontairement : modifier le premier changerait l'URL des projets déjà créés
Aucune interface graphique	L'usage passe par la ligne de commande ou le formulaire GitLab. Le portail développeur reste à l'état de plan
Périmètre d'exclusion unique	Un seul groupe peut être exclu de l'audit, par préfixe de chemin. Une liste d'exclusions serait plus souple

5.5 Perspectives d'évolution

5.5.1 Court et moyen terme

Quatre actions sont immédiatement prioritaires : **confirmer une exécution intégralement verte** de la chaîne GOVERNIX avec correction automatique activée, après application de tous les correctifs ; **trancher la question du niveau d'accès des tags protégés** en relisant l'état stocké par GitLab immédiatement après une correction réussie ; **étendre les tests unitaires** aux deux régressions identifiées mais non couvertes (modélisation des niveaux d'accès et détection dynamique de la branche par défaut) ; et **nettoyer le code résiduel**, une interface obsolète et une méthode orpheline subsistant sans appelant.

À moyen terme, quatre évolutions se dégagent. L'**externalisation du référentiel de règles** dans un fichier de configuration est la plus structurante : aujourd'hui, les niveaux attendus sont des constantes du code, et une évolution du référentiel impose une modification de code et un redéploiement ; un fichier YAML validé par schéma permettrait à l'équipe DevOps d'ajuster les règles sans intervention de développement. Viennent ensuite la **phase zéro du portail développeur**, en respectant le principe de non-duplication ; la **conteneurisation de l'outil d'audit** avec analyse de vulnérabilités intégrée ; et l'ajout d'une **notification** des écarts vers un canal d'équipe, afin que le rapport soit poussé plutôt que consulté.

5.5.2 Long terme

Trois directions se dessinent à plus longue échéance. L'**historisation et le suivi de tendance** — conserver les rapports dans un stockage durable et produire un indicateur de conformité du parc dans le temps — transformeraient l'outil de contrôle ponctuel en instrument de pilotage. L'**exécution planifiée orchestrée**, sous forme de tâche récurrente dans un ordonnanceur de conteneurs avec gestion des secrets par coffre-fort, industrialiserait le déploiement. Enfin, l'**extension à d'autres technologies** : le mécanisme de détection repose aujourd'hui sur deux manifestes, et une conception en registre extensible, envisagée puis explicitement reportée, permettrait d'ajouter d'autres écosystèmes sans modifier la logique des règles.

5.6 Apports personnels

5.6.1 Compétences techniques

Le stage a été l'occasion d'acquérir, en partant de zéro sur plusieurs de ces sujets : la **programmation Python** appliquée à un cas réel (structuration en modules, gestion des exceptions, parallélisme par fils d'exécution) ; la **consommation d'une API REST** (authentification, pagination, encodage des identifiants, interprétation des codes de retour, et surtout lecture critique d'une documentation) ; l'**intégration continue** (chaîne multi-étapes, artefacts, transmission d'état entre jobs, optimisation des temps d'exécution) ; la **maîtrise de Git** au-delà des commandes courantes (copie miroir, récupération d'un commit orphelin par le journal des références) ; les **tests unitaires** avec simulacres, et la compréhension qu'un simulacre trop permissif ne teste rien ; enfin l'**administration d'une plateforme** auto-hébergée, du déploiement en conteneur au diagnostic réseau.

5.6.2 Compétences méthodologiques

Les apports non techniques sont au moins aussi importants.

Traçabilité des décisions. Tenir un document de décisions daté, avec mention explicite des décisions remplacées, a permis de détecter plusieurs écarts entre ce qui était supposé fait et ce qui l'était réellement.

Vérifier plutôt que supposer. La leçon la plus répétée du stage. Un correctif supposé appliqué ne l'était pas. Une suite de tests supposée committée ne l'était pas. Un fichier supposé sauvegardé ne l'était pas. Une écriture supposée effective ne l'était pas. À chaque fois, une vérification de dix secondes aurait évité des heures d'investigation sur une hypothèse fausse.

Échouer bruyamment et dialoguer. Le principe *fail-closed* n'est pas une préférence esthétique : un outil qui devine produit une confiance injustifiée, ce qu'un outil de conformité ne doit jamais faire. Par ailleurs, plusieurs décisions structurantes sont nées d'une question posée à l'encadrant plutôt que d'une hypothèse retenue — la confirmation qu'un type de projet correspond à exactement une technologie a permis de supprimer toute une logique d'arbitrage devenue inutile, soit une réduction nette de complexité obtenue par une simple question.

Conclusion du chapitre

Les deux outils atteignent leurs objectifs, avec une réserve honnêtement documentée sur la validation finale de bout en bout de la chaîne de provisionnement. Les bugs rencontrés, loin d'être de simples contretemps, ont été la principale source d'apprentissage : ils ont tous porté sur l'écart entre le modèle mental du développeur et le comportement réel du système, qui est précisément la difficulté centrale de l'intégration de systèmes.

Conclusion générale

Ce projet de fin d'année est parti d'un constat simple et largement partagé dans les organisations qui produisent du logiciel : des règles de gouvernance existent, elles sont documentées, elles font consensus — et personne ne dispose des moyens de vérifier qu'elles sont respectées. Entre la règle écrite et la configuration réelle d'un dépôt, il n'y avait aucun mécanisme de contrôle, seulement de la bonne volonté et de la mémoire.

Deux outils complémentaires ont été développés pour combler cet écart.

Audit-Tool apporte le contrôle. Il parcourt l'ensemble des dépôts d'une instance GitLab, évalue cinq règles de conformité portant sur les protections de branches, les protections de tags, la contrainte de validation par la chaîne d'intégration et les accès d'un utilisateur de service, puis produit un rapport consolidé dans trois formats et historisé par exécution. Il ne s'arrête pas au constat : une couche de remédiation corrige les écarts détectés, avec deux stratégies distinctes selon l'édition de GitLab — une nécessité découverte en cours de développement, l'API n'offrant pas les mêmes possibilités de modification selon la licence. L'ensemble s'exécute en intégration continue, manuellement ou par planification, ce qui transforme un contrôle ponctuel en surveillance périodique.

GovernX apporte la prévention. Constatant qu'un outil qui corrige *a posteriori* livre un combat sans fin, il déplace le contrôle au moment de la création du projet. Une chaîne CI/CD en trois étapes résout un dépôt modèle par *topic* GitLab, le recopie intégralement vers un ou plusieurs nouveaux projets, aligne l'identité du projet à l'intérieur même de son manifeste de build, applique les *topics* de classification, puis réutilise les règles d'Audit-Tool pour auditer et corriger le résultat. Un projet créé par cette chaîne est conforme parce que la conformité fait partie de sa création, non parce que quelqu'un a pensé à la configurer.

Au-delà des livrables, ce travail a produit trois convictions que je retiens.

La première est qu'un outil de conformité qui se trompe est pire qu'un outil absent. Le bug le plus grave rencontré durant le stage ne provoquait aucune erreur visible : il créait une protection sur une branche inexistante, la re-vérification la trouvait bien présente, et le système déclarait le projet conforme pendant que la véritable branche principale restait sans aucune protection. Un outil absent laisse une incertitude connue ; un outil qui se trompe crée une certitude fausse. C'est ce qui justifie le principe *fail-closed* appliqué systématiquement : devant une ambiguïté, échouer bruyamment plutôt que deviner.

La deuxième est qu'il faut vérifier au lieu de supposer. Cette leçon s'est répétée sous des formes différentes tout au long du stage : un correctif supposé appliqué qui ne l'était pas, une suite de tests supposée versionnée qui n'existait nulle part, une écriture supposée effective que le serveur avait silencieusement ignorée. À chaque fois, une vérification élémentaire aurait évité des heures d'investigation menée sur une hypothèse fausse.

La troisième est que la duplication de logique métier est une dette immédiate. La tentation était réelle de réimplémenter les règles dans la chaîne de provisionnement plutôt que de les réutiliser. Deux implémentations d'une même règle divergent au premier correctif appliqué d'un seul côté, et la divergence est silencieuse. Ce principe a été rappelé explicitement chaque fois qu'une nouvelle couche était envisagée, y compris pour le portail développeur resté à l'état de plan.

Sur le plan personnel, ce stage a représenté une transformation notable. Débuté sans expérience préalable de Python ni de l'API GitLab, il a permis d'acquérir non seulement des compétences techniques — programmation, consommation d'API, intégration continue, tests, administration de plateforme — mais surtout une méthode : cadrer avant de coder, tracer les décisions et leurs raisons, tester ce qui peut l'être, et documenter honnêtement ce qui reste incertain. Les bugs les plus coûteux n'ont pas été des erreurs de syntaxe mais des écarts entre ce que je croyais que le système faisait et

ce qu'il faisait réellement. Apprendre à repérer et à réduire cet écart est probablement l'acquis le plus durable de ces semaines.

Les deux outils sont fonctionnels et validés sur l'environnement de développement. Leur mise en production suppose des arbitrages qui relèvent de la Direction des Systèmes d'Information — droits du compte de service, périmètre d'application, politique de correction automatique. Les perspectives identifiées, de l'externalisation du référentiel de règles au portail développeur en libre-service, dessinent une trajectoire cohérente vers une gouvernance des dépôts entièrement automatisée, où la conformité cesse d'être une contrainte subie pour devenir une propriété garantie par construction.

Annexes

A.1 Audit-Tool — options de la ligne de commande

Table A.1 – Options de la ligne de commande d’Audit-Tool

Option	Statut	Rôle
<code>--url</code>	obligatoire	URL de l’instance GitLab à auditer
<code>--token</code>	obligatoire	Jeton de scope <code>api</code>
<code>--user-id</code>	optionnel	Identifiants ou noms d’utilisateurs séparés par des virgules; active la règle d’accès utilisateur
<code>--fix</code>	drapeau	Active la remédiation automatique
<code>--dry-run</code>	drapeau	Simule sans écrire (nécessite <code>--fix</code>)
<code>--tier</code>	défaut <code>free</code>	<code>free</code> (Community) ou <code>premium</code> (Enterprise)
<code>--user-branches</code>	optionnel	Branches et types d’accès, ex. <code>"main:both,develop:merge"</code>

```

1 # Audit en lecture seule / simulation / correction réelle
2 python -m audit_tool.main --url http://gitlab.local --token "$TOKEN"
3 python -m audit_tool.main --url http://gitlab.local --token "$TOKEN" --fix --dry-run
4 python -m audit_tool.main --url http://gitlab.local --token "$TOKEN" --fix
5
6 # Edition Premium, acces differencie par branche
7 python -m audit_tool.main --url http://gitlab.local --token "$TOKEN" \
8     --user-id alice,bob --tier premium \
9     --user-branches "main:both,develop:merge,release/*:push"
10
11 # --- Construction conditionnelle des arguments cote CI ---
12 set -- --url "$GITLAB_AUDIT_URL" --token "$GITLAB_AUDIT_TOKEN" \
13     --tier "${AUDIT_TIER:-free}"
14 if [ -n "${GITLAB_AUDIT_USER_ID:-}" ]; then
15     set -- "$@" --user-id "$GITLAB_AUDIT_USER_ID"
16 fi
17 if [ "${AUDIT_FIX:-false}" = "true" ]; then
18     set -- "$@" --fix
19     # dry-run par défaut : activer --fix seul ne doit jamais écrire
20     if [ "${AUDIT_DRY_RUN:-true}" = "true" ]; then set -- "$@" --dry-run; fi
21 fi
22 python -m audit_tool.main "$@"

```

Extrait A.1 – Exemples d’invocation et construction des arguments en CI

A.2 GovernX — variables de la chaîne CI/CD

Table A.2 – Variables de déclenchement (formulaire « Run pipeline »)

Variable	Statut	Rôle
<code>PROJECT_NAME</code>	obligatoire	Nom(s) du/des projet(s), séparés par des virgules, points-virgules ou retours à la ligne
<code>PROJECT_TYPE</code>	obligatoire	<code>backend</code> , <code>frontend</code> , <code>batch</code> ou <code>dependencie</code> ; sert de <i>topic</i> de résolution
<code>GROUP_PATH</code>	obligatoire	Chemin du groupe cible
<code>TEMPLATE_VARIANT</code>	optionnel	Désambiguïsation en cas de correspondances multiples
<code>MAX_PARALLEL_PROJECTS</code>	défaut 4	Degré de parallélisme du traitement par lot
<code>AUDIT_AUTO_FIX</code>	défaut <code>no</code>	Active la correction automatique
<code>AUDIT_ACCEPT_NONCOMPLIANT</code>	défaut <code>no</code>	Accepte un projet non conforme sans bloquer la chaîne

Quatre variables sont en outre transmises d’une étape à l’autre par artefact `dotenv` : `NEW_PROJECT_PATHS` (chemins des projets créés, source de vérité des étapes 2 et 3), `NEW_PROJECT_COUNT`, ainsi que `NEW_PROJECT_PATH` et `NEW_PROJECT_HTTP_URL`, conservées pour la compatibilité des exécutions mono-projet.

Table A.3 – Variables d’infrastructure (niveau groupe, masquées et protégées)

Variable	Statut	Rôle
TEMPLATE_GROUP_PATH	obligatoire	Groupe racine contenant les dépôts modèles
TEMPLATE_GITLAB_TOKEN	obligatoire	Jeton de lecture seule sur l’instance des modèles
TEMPLATE_GITLAB_URL	défaut gitlab.com	Instance hébergeant les modèles
GITLAB_API_TOKEN	obligatoire	Jeton de l’instance locale : API, poussée et écriture sur les protections
AUDIT_TIER	défaut free	Stratégie de correction des branches protégées
GITLAB_API_TIMEOUT	défaut 30	Délai maximal des appels d’API (secondes)

A.3 Chaîne CI/CD et arborescence de GovernX

```

1  stages: [create_project, configure_tags, audit_check]
2
3  variables:
4    PROJECT_TYPE: { value: "backend",
5                    options: ["backend", "frontend", "batch", "dependencie"] }
6    GIT_DEPTH: "1" # le depot de la chaine n'a pas besoin d'historique
7    PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"
8
9  workflow:
10   rules:
11     - if: '$CI_PIPELINE_SOURCE == "web"' # declenchement manuel uniquement
12
13  default:
14    image: python:3.11 # image complete : git deja installe (~30 s gagnees)
15    cache:
16      key: { files: [ci/requirements.txt] }
17      paths: [.cache/pip]
18    before_script: [pip install --quiet -r ci/requirements.txt]
19
20  create_project:
21    stage: create_project
22    script: [python3 -m ci.create_project]
23    artifacts:
24      when: always
25      paths: [reports/created_projects.json]
26      reports: { dotenv: create_project.env } # transmet NEW_PROJECT_PATHS
27
28  audit_check:
29    stage: audit_check
30    needs: [{ job: create_project, artifacts: true }, { job: configure_tags }]
31    script: [python3 -m ci.audit_check]
32    artifacts:
33      when: always
34      expose_as: "Rapport d'audit GovernX"
35      paths: [reports/audit_report.html, reports/audit_report.json]
36
37  # --- Structure du depot -----
38  # governx-pipeline/
39  # |-- ci/      create_project.py, configure_tags.py, audit_check.py
40  # |            lib/gitlab_api.py (wrapper REST v4), lib/projects.py
41  # |-- audit_tool/ manifest.py, gitlab_client.py, report_html.py,
42  # |                rules/{protected_branches,protected_tags,pipeline_success}.py
43  # +-- tests/    conftest.py + 3 modules, sans dependance reseau

```

Extrait A.2 – Extrait du fichier .gitlab-ci.yml et structure du dépôt

A.4 Glossaire

Table A.4 – Glossaire des termes techniques

Terme	Définition
Artefact	Fichier produit par un job d'intégration continue et conservé par la plateforme pour être téléchargé ou réutilisé
Branche protégée	Règle restreignant qui peut pousser des commits ou fusionner vers une branche ou un motif de branches
Copie miroir	Clone Git incluant toutes les références (branches, tags), par opposition à un clone simple limité à la branche par défaut
<i>Fail-closed</i>	Comportement consistant à refuser d'agir et à signaler une erreur en cas d'ambiguïté, plutôt qu'à choisir une valeur par défaut
Manifeste de build	Fichier décrivant l'identité et les dépendances d'un projet : <code>pom.xml</code> pour Maven, <code>package.json</code> pour npm
<i>Runner</i>	Agent exécutant les jobs d'une chaîne d'intégration continue
Simulacre	Objet de test imitant une dépendance réelle pour permettre un test sans accès externe
Tag protégé	Règle restreignant qui peut créer un tag correspondant à un motif
<i>Topic</i>	Étiquette libre attachée à un projet, utilisée ici comme mécanisme de résolution de modèle

Webographie et références

Documentation officielle GitLab

- [1] **REST API — Protected branches** : points d'entrée, champs `push_access_level` / `merge_access_level` et restrictions d'édition sur `allowed_to_push` / `allowed_to_merge`. https://docs.gitlab.com/ee/api/protected_branches.html
- [2] **REST API — Protected tags** : valeurs admises pour `create_access_level`; référence établissant l'absence de méthode de modification en place. https://docs.gitlab.com/ee/api/protected_tags.html
- [3] **REST API — Projects** : création, *topics*, attribut `only_allow_merge_if_pipeline_succeeds`. <https://docs.gitlab.com/ee/api/projects.html>
- [4] **REST API — Repositories et Commits** : lecture d'arborescence, fichiers bruts, création de commits sans clone local. <https://docs.gitlab.com/ee/api/repositories.html>
- [5] **REST API — Pagination** : en-tête `X-Next-Page`. <https://docs.gitlab.com/ee/api/rest/index.html#pagination>
- [6] **Members and roles** : niveaux d'accès et permissions associées. <https://docs.gitlab.com/ee/user/permissions.html>
- [7] **CI/CD — Référence .gitlab-ci.yml** : *stages*, *jobs*, *artifacts*, *needs*, *workflow:rules*. <https://docs.gitlab.com/ee/ci/yaml/>
- [8] **CI/CD — Variables** : portées, attributs « masquée » et « protégée », artefacts `dotenv`. <https://docs.gitlab.com/ee/ci/variables/>

Bibliothèques et outils

- [9] **python-gitlab** — client Python de l'API GitLab. <https://python-gitlab.readthedocs.io/>
- [10] **Requests** — bibliothèque HTTP utilisée par le client REST de GOVERNIX. <https://requests.readthedocs.io/>
- [11] **pytest** — cadre de test de la suite unitaire. <https://docs.pytest.org/>
- [12] **Git** — `clone -mirror`, `push -mirror`, `reflog`, `cherry-pick`. <https://git-scm.com/docs>
- [13] **Trivy** — analyseur de vulnérabilités d'images de conteneurs (axe 3). <https://trivy.dev/>
- [14] **Backstage** — portail développeur et *software templates* (axe 2). <https://backstage.io/docs/features/software-templates/>
- [15] **Maven — POM Reference** : sémantique de `<artifactId>`, `<groupId>`, `<parent>`. <https://maven.apache.org/pom.html>
- [16] **npm — package.json** : contraintes de nommage du champ `name`. <https://docs.npmjs.com/cli/configuring-npm/package-json>

Normes et cadre réglementaire

- [17] **ISO/CEI 27001** — Systèmes de management de la sécurité de l'information. Les exigences de contrôle d'accès et de gestion des changements sous-tendent les règles de gouvernance implémentées.
- [18] **PCI DSS** — *Payment Card Industry Data Security Standard* : séparation des rôles et contrôle des modifications en production.
- [19] **Loi n° 09-08** relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (Royaume du Maroc) — cadre de référence pour l'analyse d'une éventuelle externalisation vers un cloud public.

Documentation interne du projet

- [20] **GovernX.md** — registre daté des quinze décisions d’architecture de GOVERN X, avec justifications, décisions remplacées et journal des changements.
- [21] **projet_audit_gitlab_cih.md** — document de cadrage d’Audit-Tool : périmètre des règles, conventions de format, architecture par couches, comptes rendus de sessions.
- [22] **README.md** des deux dépôts — documentation d’exploitation : structure, variables, procédure d’exécution des tests.